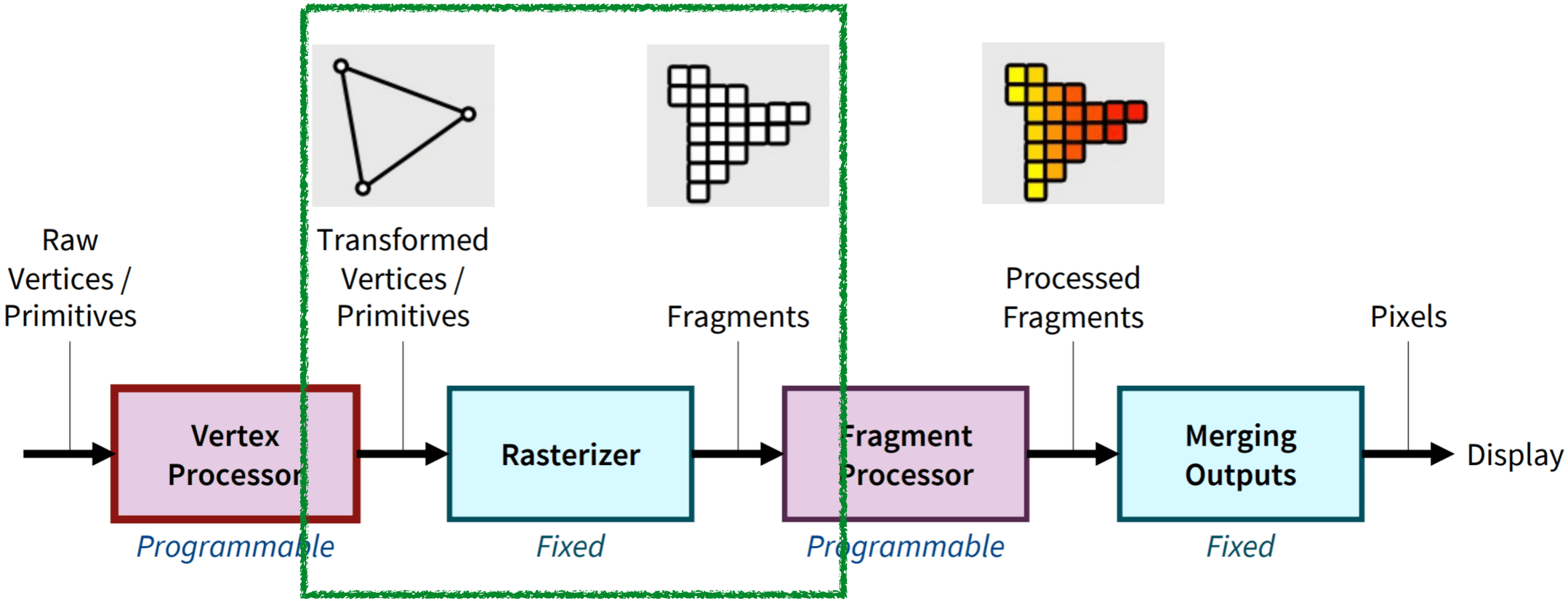
Rasterization

Ray tracing → Rasterization



Youngdo Lee; https://leeyngdo.github.io/blog/computer-graphics/2024-02-29-graphics-pipeline/

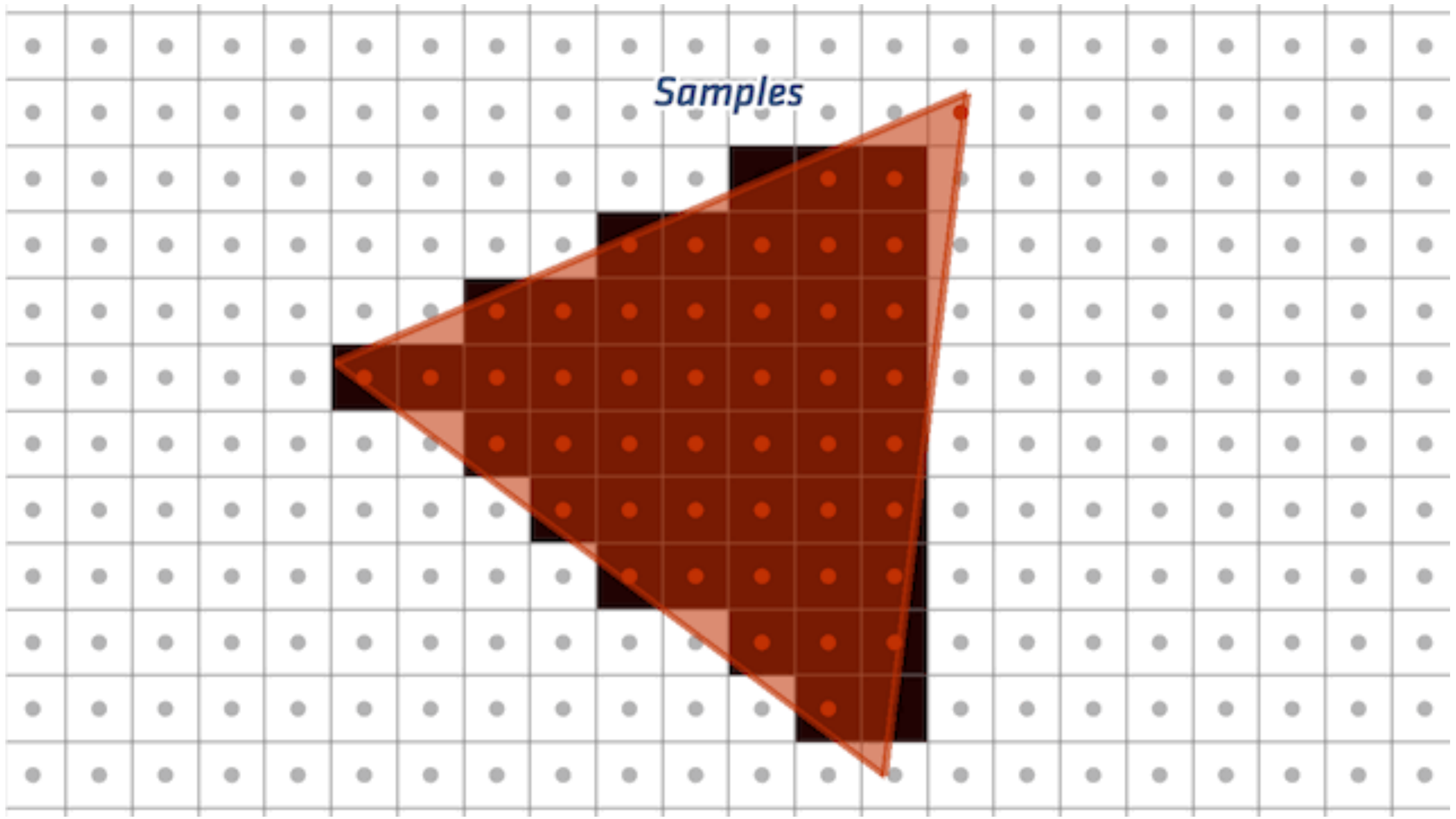


Image Copyright: Andrea Tagliasacchi

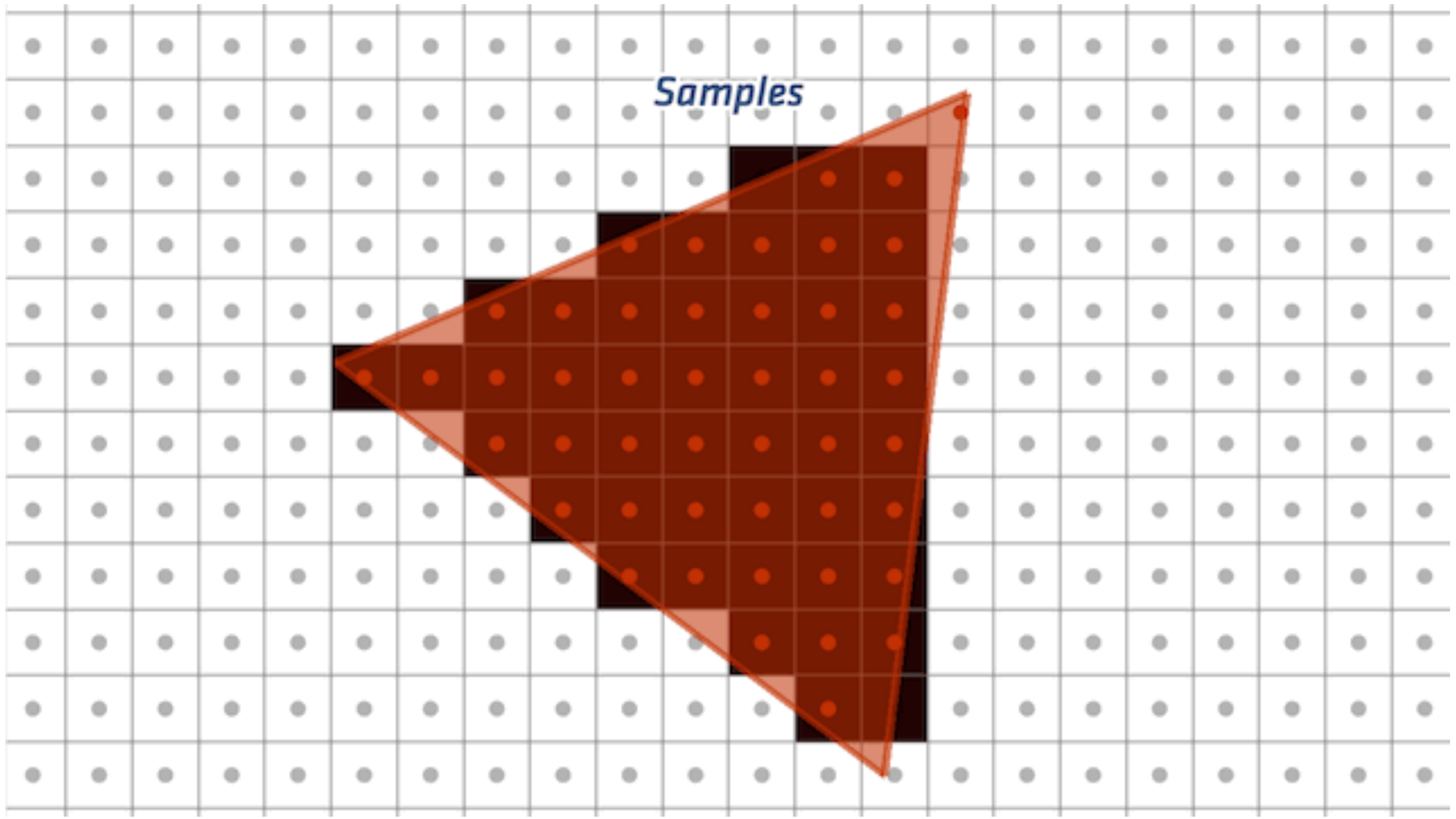
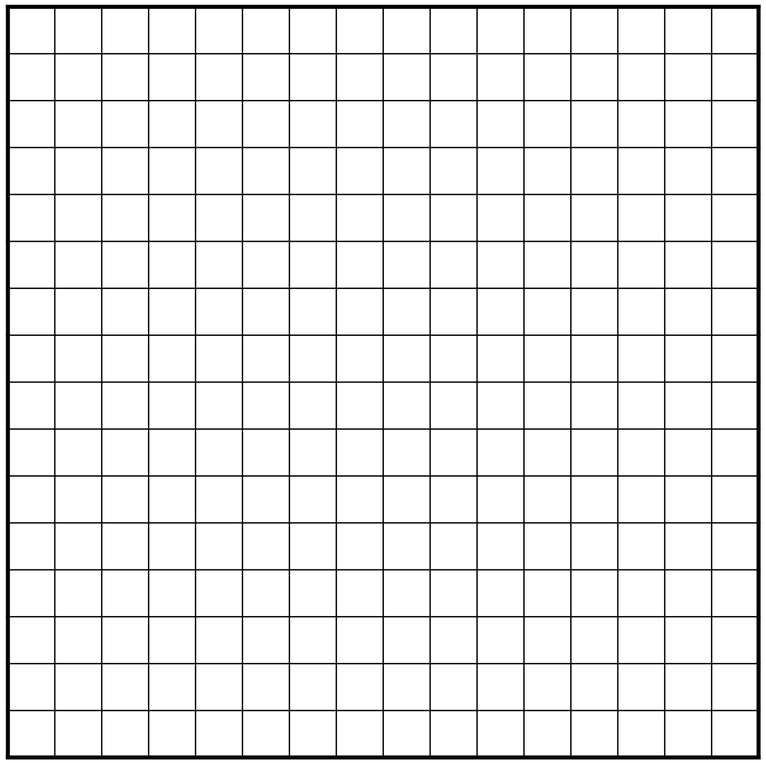


Image Copyright: Andrea Tagliasacchi

2D Canvas



pixel grid

Image Copyright: Andrea Tagliasacchi

2D Canvas

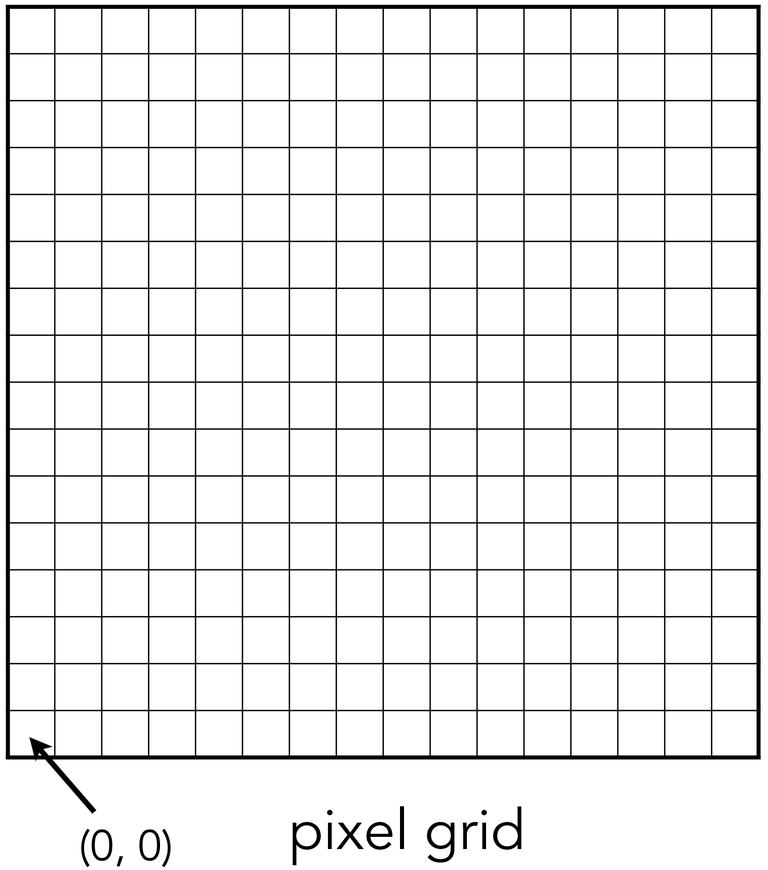


Image Copyright: Andrea Tagliasacchi

2D Canvas

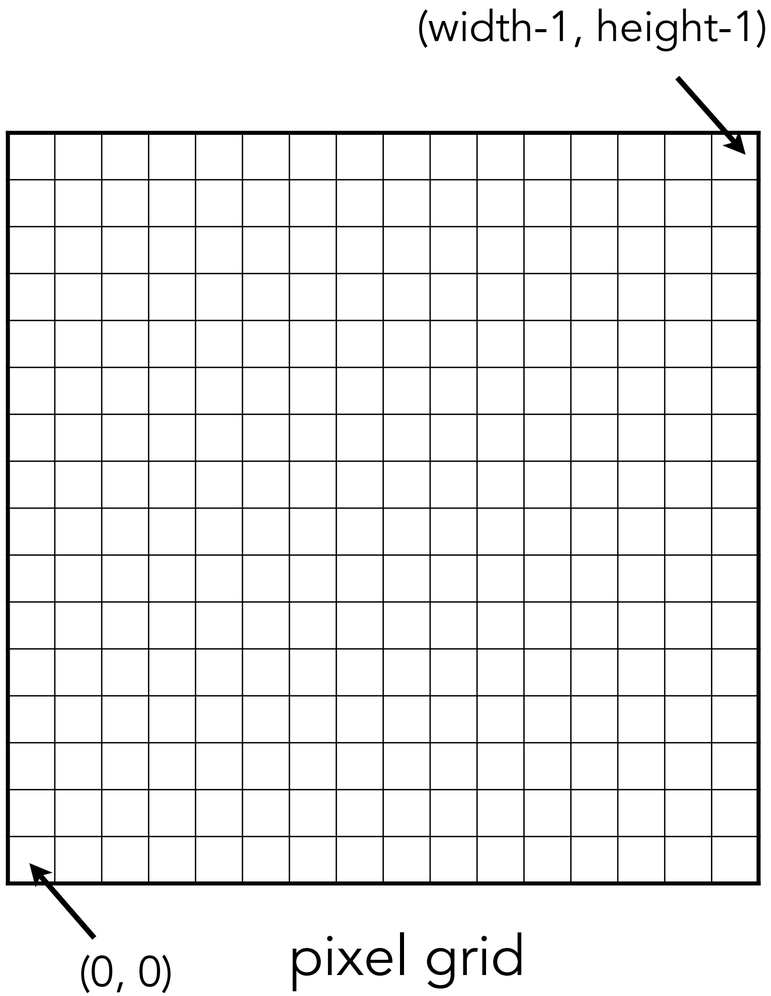


Image Copyright: Andrea Tagliasacchi

2D Canvas

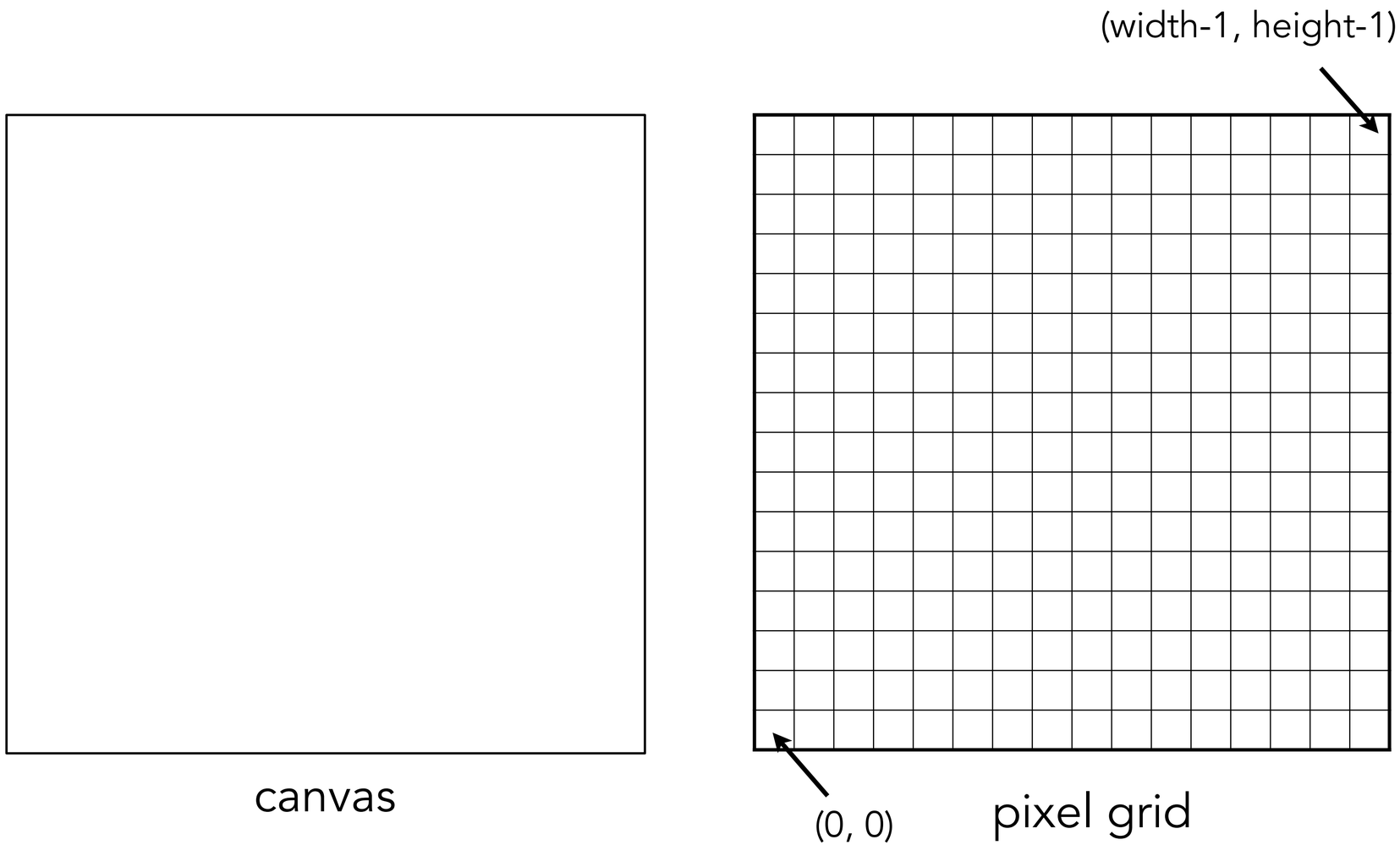


Image Copyright: Andrea Tagliasacchi

2D Canvas

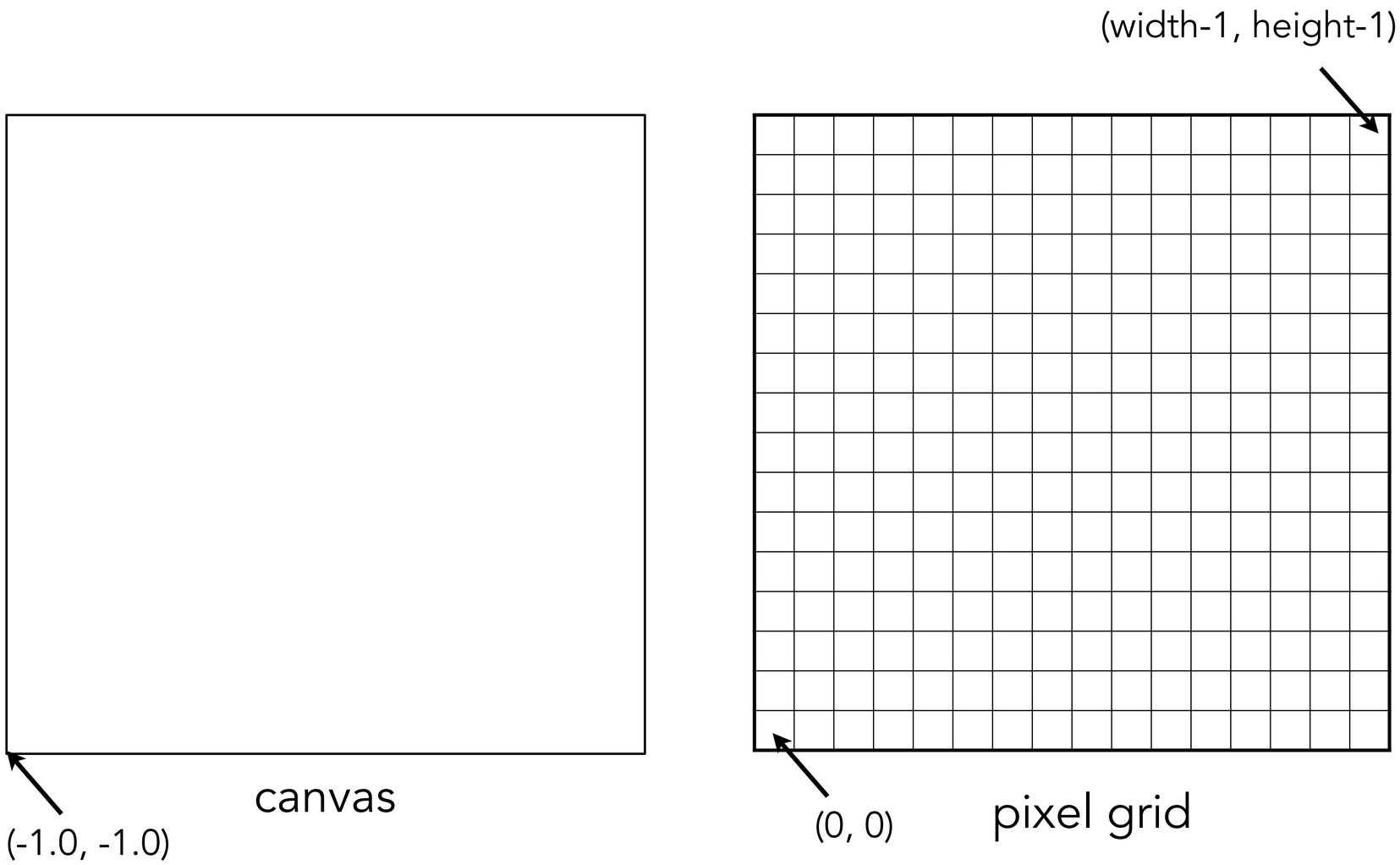


Image Copyright: Andrea Tagliasacchi

2D Canvas

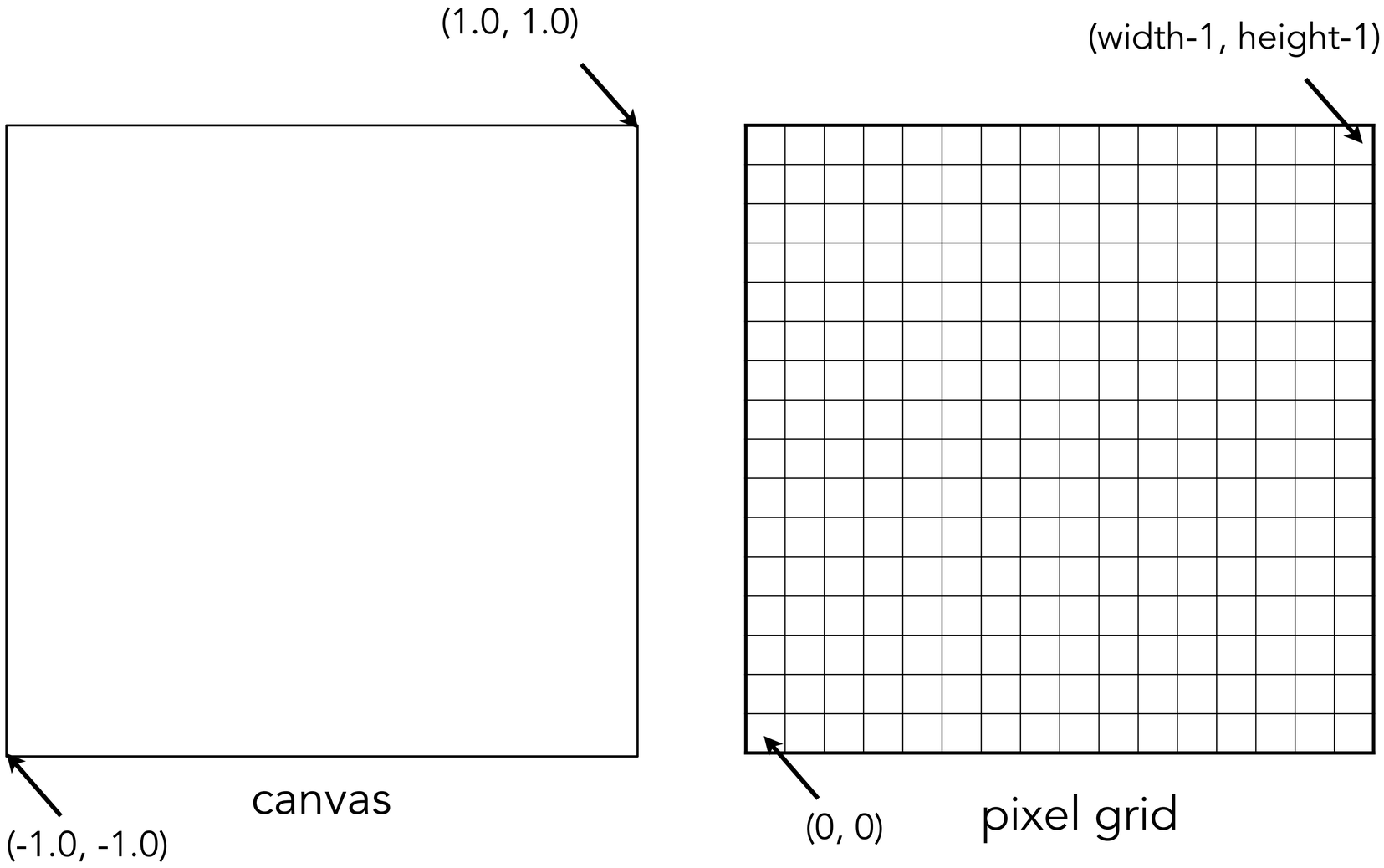


Image Copyright: Andrea Tagliasacchi

2D Canvas

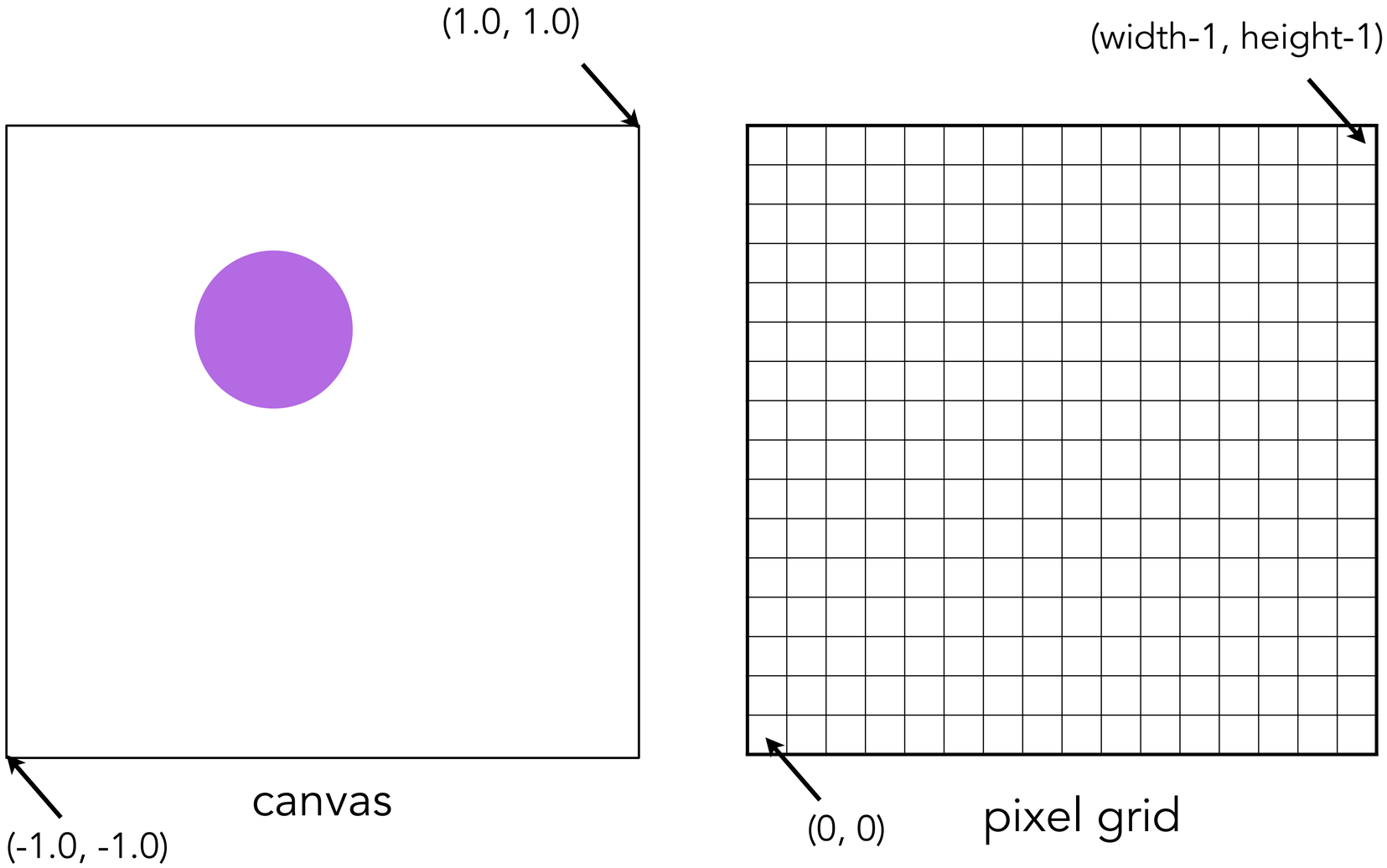


Image Copyright: Andrea Tagliasacchi

2D Canvas

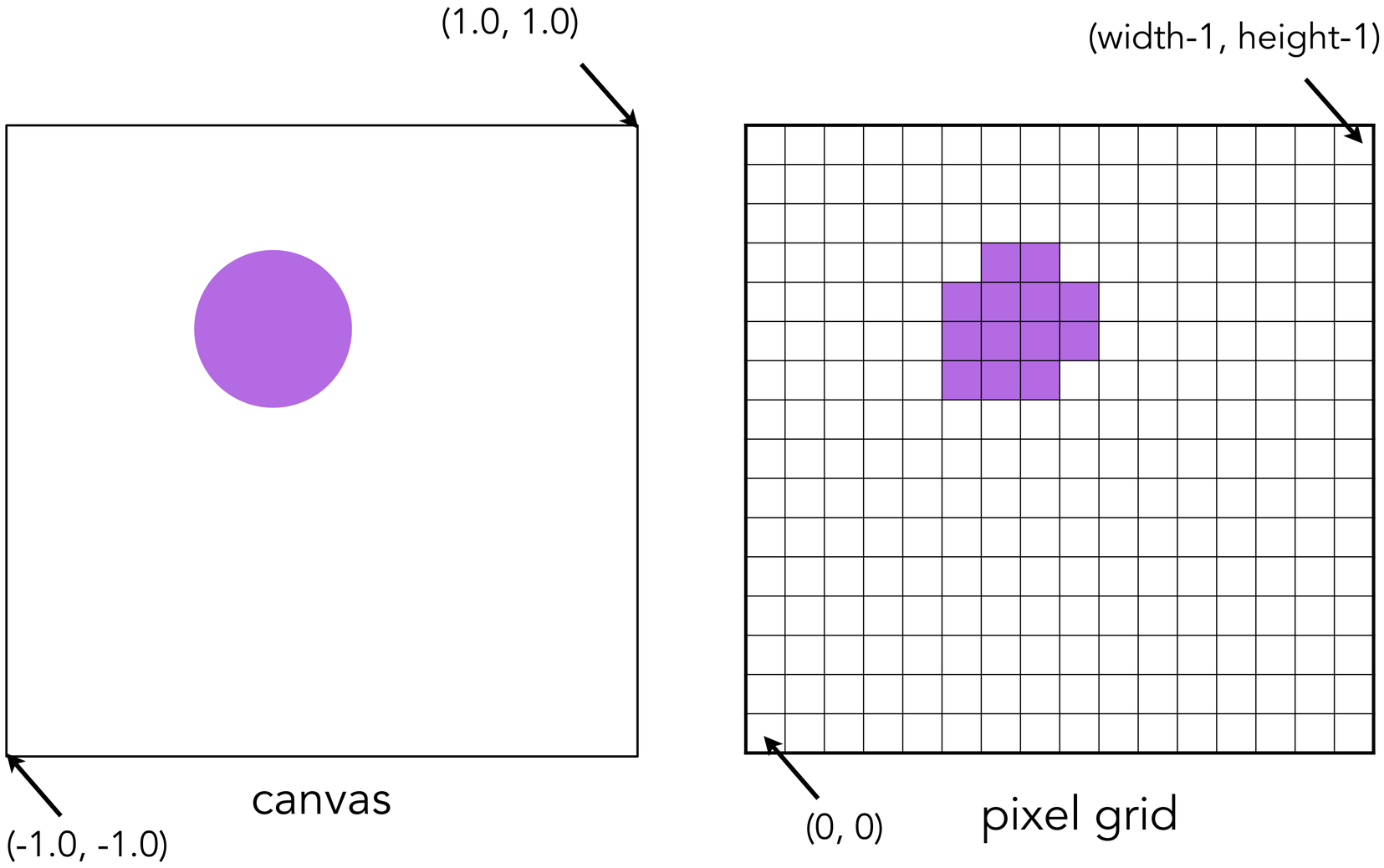


Image Copyright: Andrea Tagliasacchi

Implicit Geometry Representation

- Define a curve as zero set of 2D implicit function

University of Victoria | Computer Science

Implicit Geometry Representation

- Define a curve as zero set of 2D implicit function
  - $F(x,y) = 0 \rightarrow$ on curve

University of Victoria | Computer Science

Implicit Geometry Representation

- Define a curve as zero set of 2D implicit function
  - $F(x,y) = 0 \rightarrow$ on curve
  - $F(x,y) < 0 \rightarrow$ inside curve

University of Victoria | Computer Science

Implicit Geometry Representation

- Define a curve as zero set of 2D implicit function
  - $F(x,y) = 0 \rightarrow$ on curve
  - $F(x,y) < 0 \rightarrow$ inside curve
  - $F(x,y) > 0 \rightarrow$ outside curve

University of Victoria | Computer Science

Implicit Geometry Representation

- Define a curve as zero set of 2D implicit function
  - $F(x,y) = 0 \rightarrow$ on curve
  - $F(x,y) < 0 \rightarrow$ inside curve
  - $F(x,y) > 0 \rightarrow$ outside curve
- Example: Circle with center (c_x, c_y) and radius r

University of Victoria | Computer Science

Implicit Geometry Representation

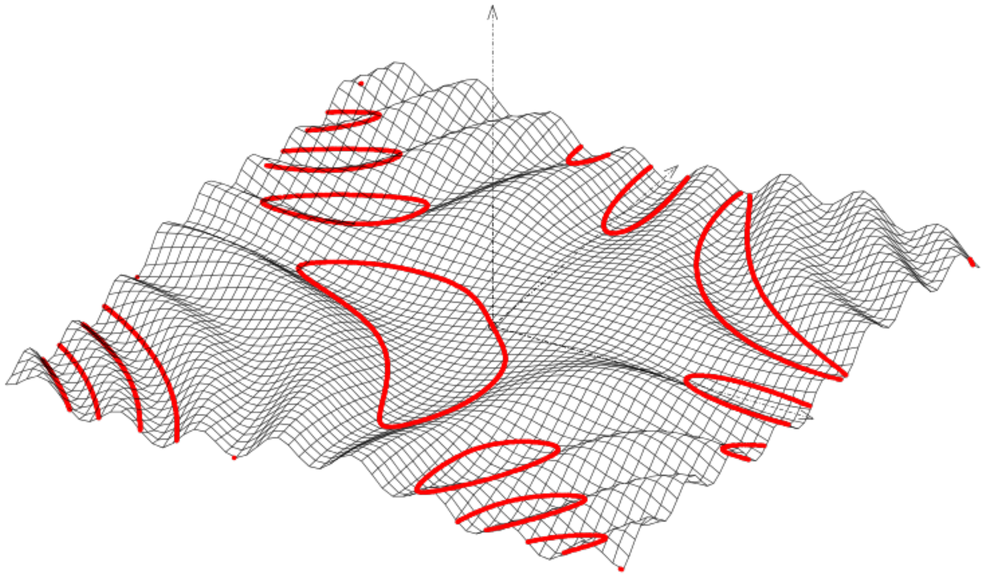
- Define a curve as zero set of 2D implicit function
  - $F(x,y) = 0$ → on curve
  - $F(x,y) < 0$ → inside curve
  - $F(x,y) > 0$ → outside curve
- Example: Circle with center (c_x, c_y) and radius r

$$F(x, y) = (x - c_x)^2 + (y - c_y)^2 - r^2$$

University of Victoria | Computer Science

Implicit Geometry Representation

- Define a curve as zero set of 2D implicit function
  - $F(x,y) = 0 \rightarrow$ on curve
  - $F(x,y) < 0 \rightarrow$ inside curve
  - $F(x,y) > 0 \rightarrow$ outside curve



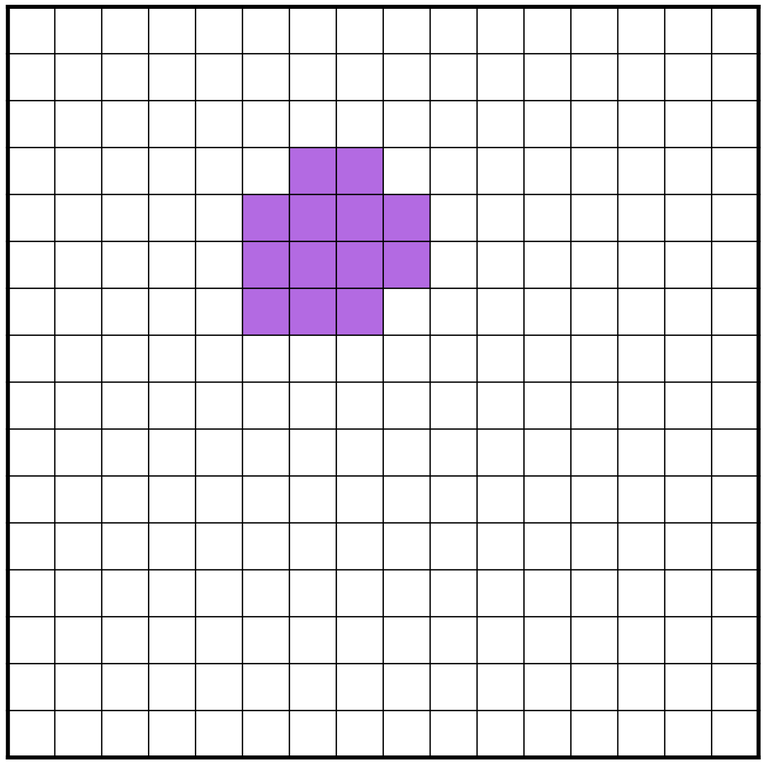
By Ag2gaeh - Own work, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=45004240

Implicit Rasterization

for all pixels (i,j)

(x,y) = map_to_canvas (i,j)

if F(x,y) < 0



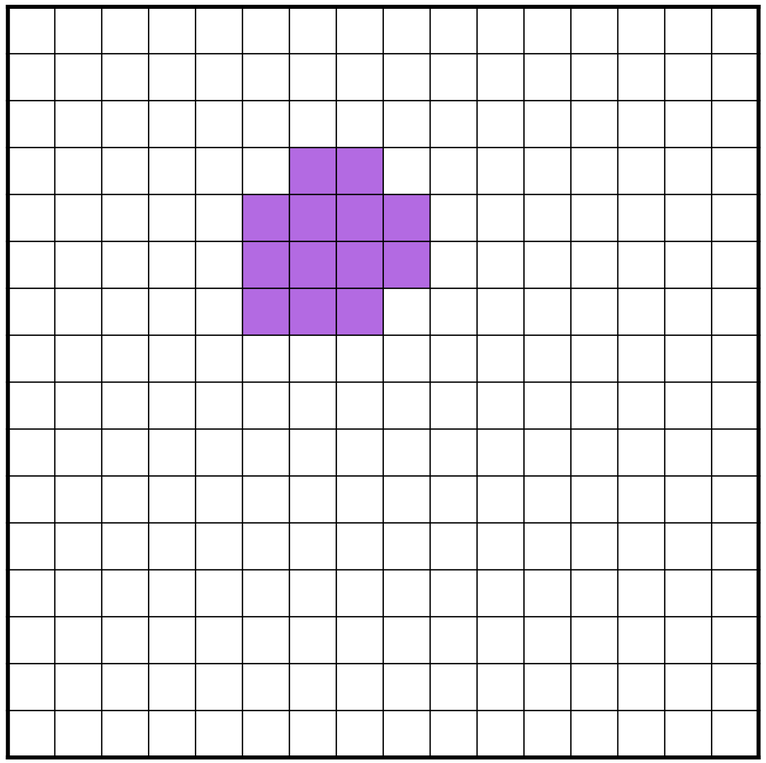
Implicit Rasterization

```
for all pixels (i,j)

(x,y) = map_to_canvas (i,j)

 if F(x,y) < 0

       set_pixel (i,j, color)
```



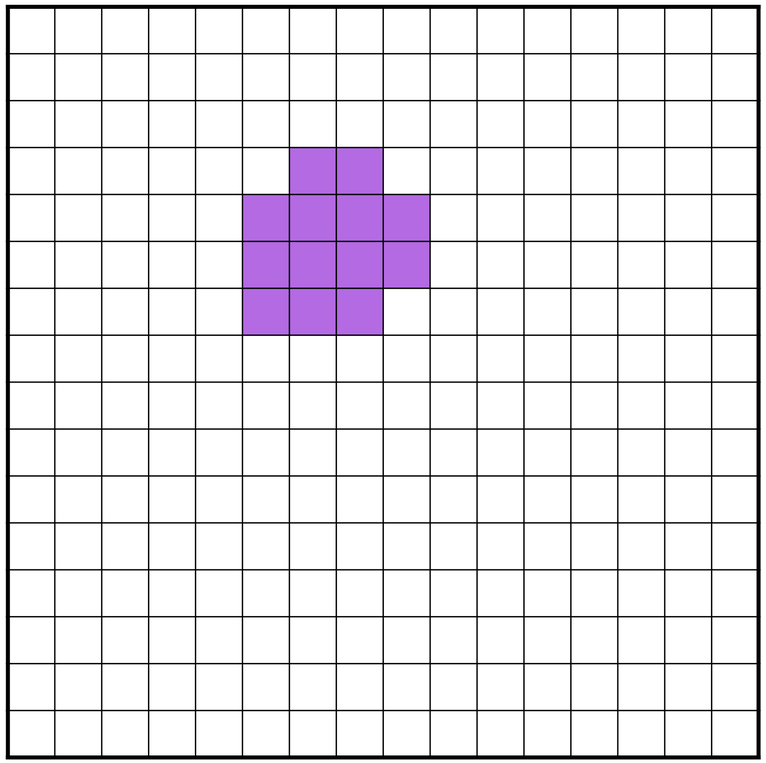
University of Victoria | Computer Science

Implicit Rasterization

for each triangle (v0, v1, v2) in screen space:

    for y in ymin .. ymax:

        for x in xmin .. xmax:



University of Victoria | Computer Science

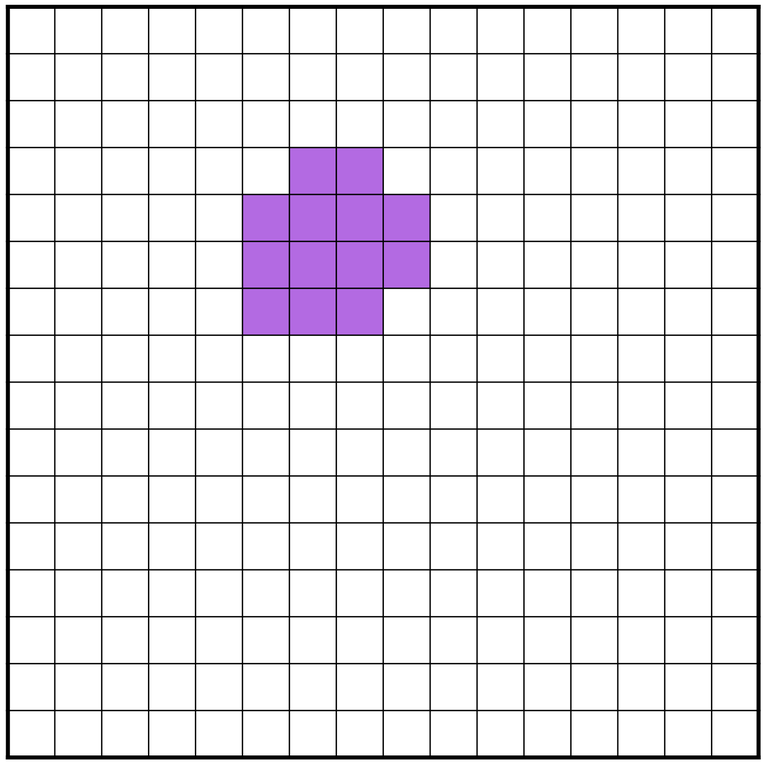
Implicit Rasterization

for each triangle (v0, v1, v2) in screen space:

    for y in ymin .. ymax:

        for x in xmin .. xmax:

            p = (x, y)



University of Victoria | Computer Science

Implicit Rasterization

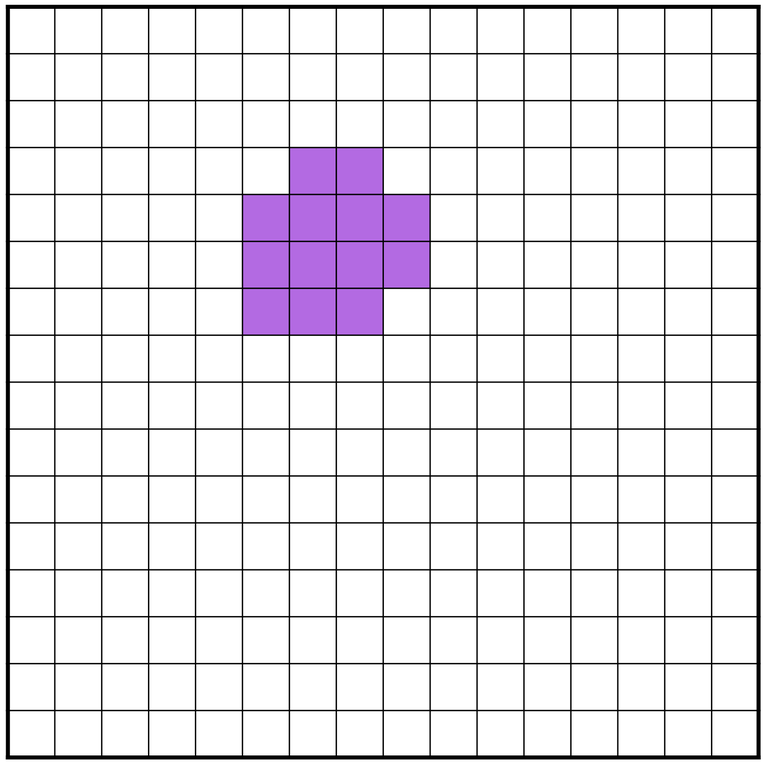
```
for each triangle (v0, v1, v2) in screen space:

    for y in ymin .. ymax:

        for x in xmin .. xmax:

            p = (x, y)

            if F(x,y) < 0
```



University of Victoria | Computer Science

Implicit Rasterization

```
for each triangle (v0, v1, v2) in screen space:

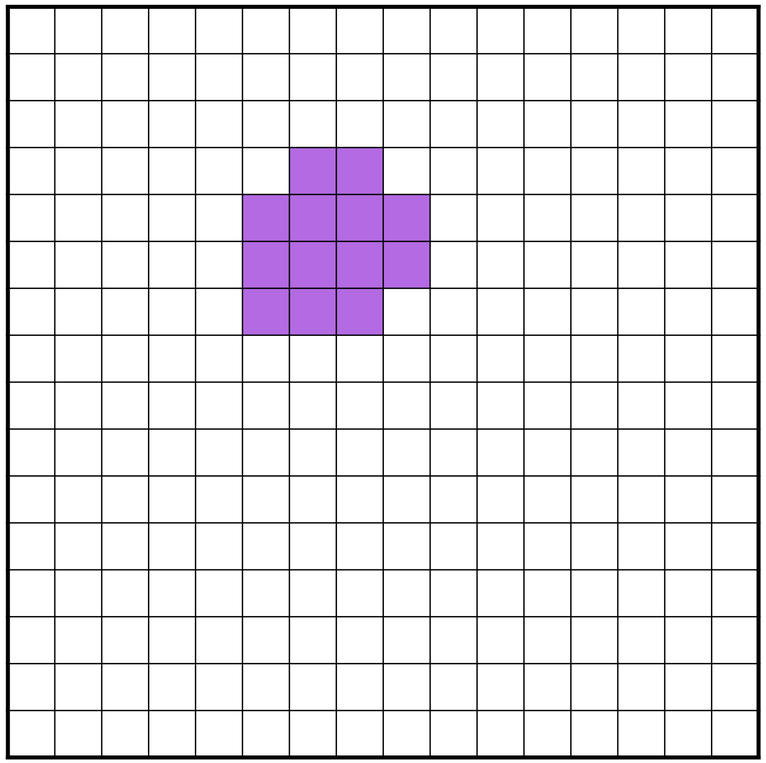
    for y in ymin .. ymax:

        for x in xmin .. xmax:

            p = (x, y)

            if F(x,y) < 0

                set_pixel (i,j, color)
```



University of Victoria | Computer Science

Implicit Rasterization

```
for each triangle (v0, v1, v2) in screen space:

    for y in ymin .. ymax:

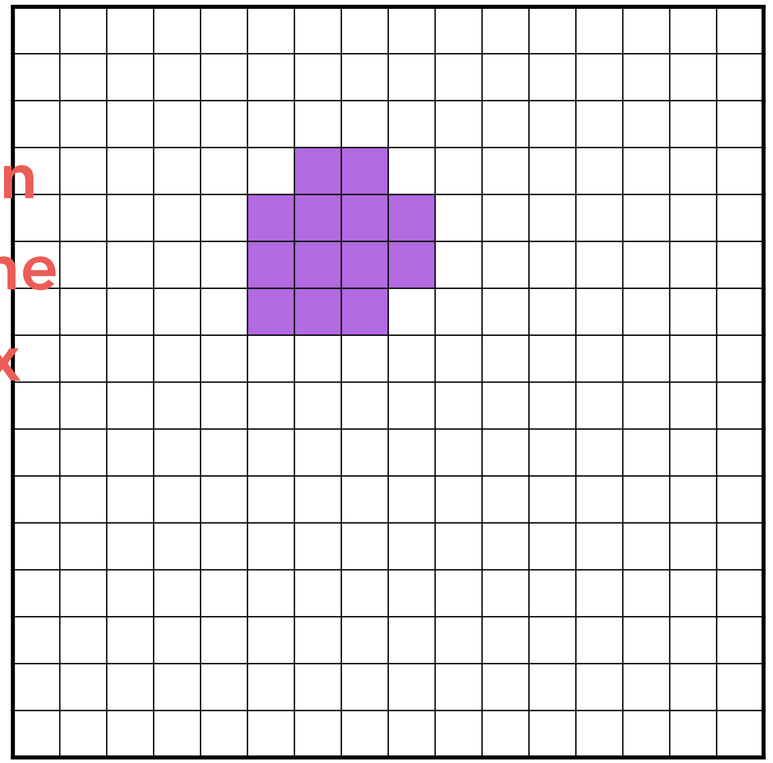
        for x in xmin .. xmax:

            p = (x, y)

            if F(x,y) < 0

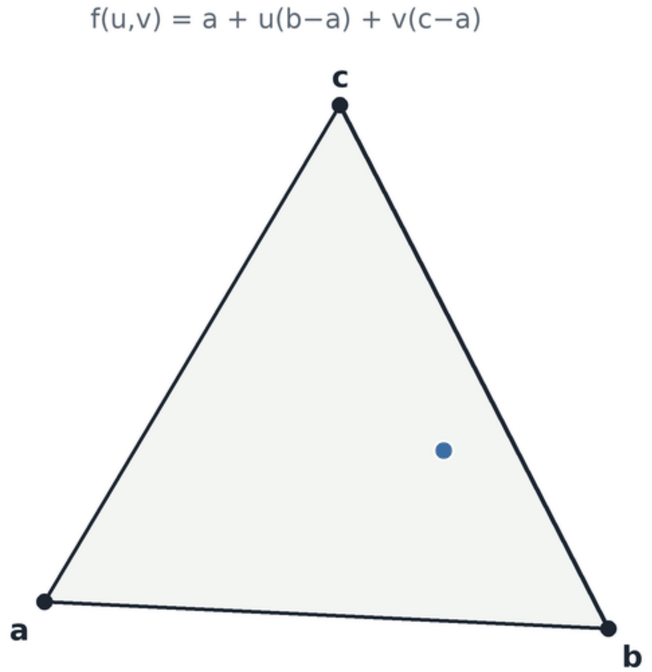
                set_pixel (i,j, color)
```

Only iterate on pixels within the bounding box



University of Victoria | Computer Science

Barycentric Interpolation

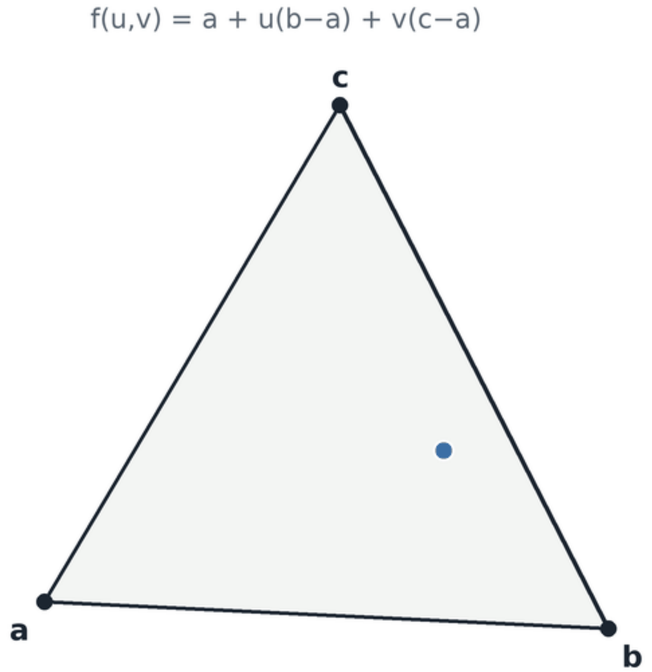


$$\mathbf{f}(u,v) = \mathbf{a} + u(\mathbf{b}-\mathbf{a}) + v(\mathbf{c}-\mathbf{a})$$

$$0 \leq u, v \quad u + v \leq 1$$

University of Victoria | Computer Science

Barycentric Interpolation

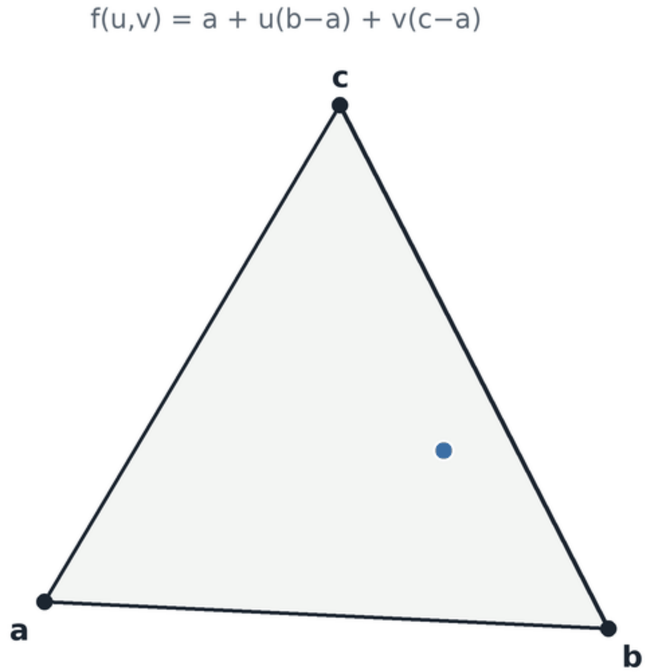


$$\mathbf{f}(u,v) = \mathbf{a} + u(\mathbf{b}-\mathbf{a}) + v(\mathbf{c}-\mathbf{a})$$

$$0 \leq u, v \quad u + v \leq 1$$

University of Victoria | Computer Science

Barycentric Interpolation

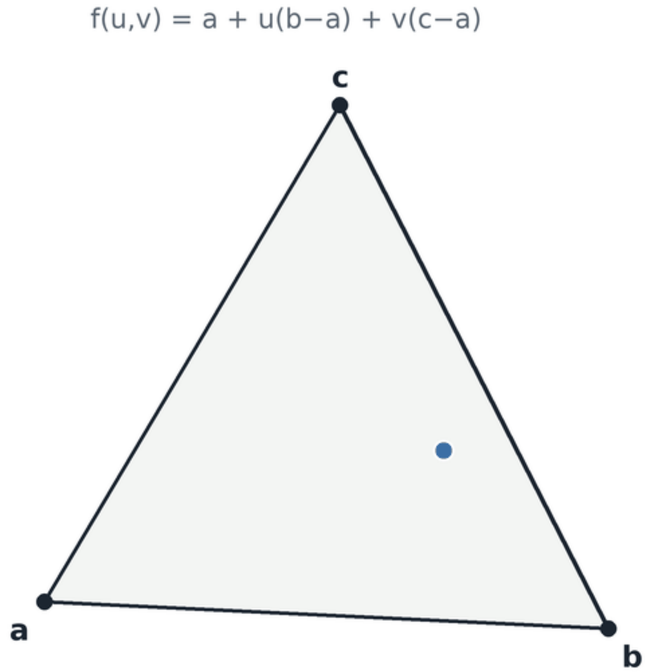


$$\mathbf{f}(u,v) = (1-u-v)\mathbf{a} + u\mathbf{b} + v\mathbf{c}$$

$$0 \leq u, v \quad u + v \leq 1$$

University of Victoria | Computer Science

Barycentric Interpolation



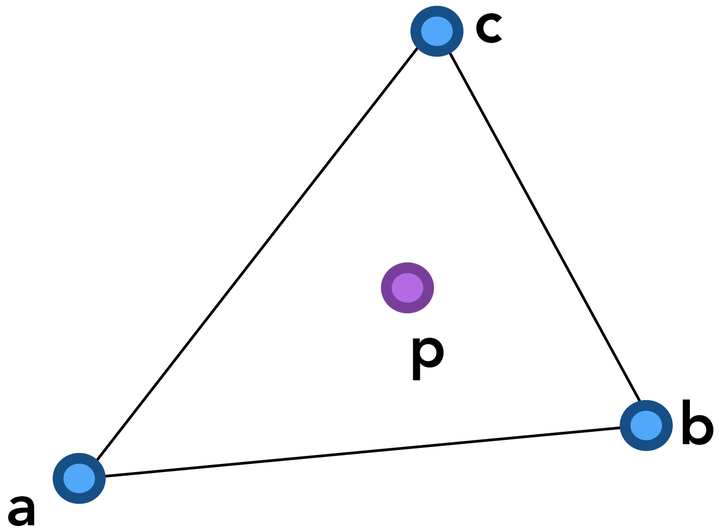
$$\mathbf{f}(u,v) = (1-u-v)\mathbf{a} + u\mathbf{b} + v\mathbf{c}$$

$$0 \leq u, v \quad u + v \leq 1$$

University of Victoria | Computer Science

Barycentric Interpolation

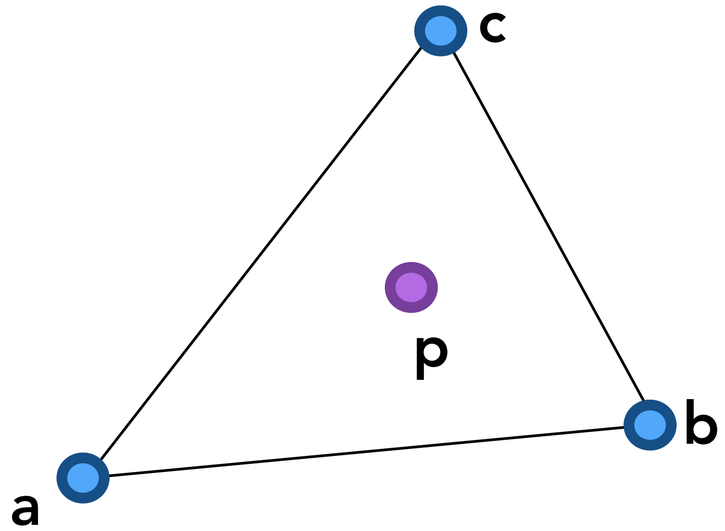
- Barycentric coordinates: $(1-u-v)+u+v=1$
  - $\mathbf{p} = \alpha\mathbf{a} + \beta\mathbf{b} + \gamma\mathbf{c}$ with $\alpha + \beta + \gamma = 1$



University of Victoria | Computer Science

Barycentric Interpolation

- Barycentric coordinates:
  - $\mathbf{p} = \alpha\mathbf{a} + \beta\mathbf{b} + \gamma\mathbf{c}$ with $\alpha + \beta + \gamma = 1$
  - Unique for non-collinear **a**,**b**,**c**



$$\begin{bmatrix} \mathbf{a}_x & \mathbf{b}_x & \mathbf{c}_x \\ \mathbf{a}_y & \mathbf{b}_y & \mathbf{c}_y \\ 1 & 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} \alpha \\ \beta \\ \gamma \end{bmatrix} = \begin{bmatrix} \mathbf{p}_x \\ \mathbf{p}_y \\ 1 \end{bmatrix}$$

University of Victoria | Computer Science

Barycentric Interpolation

- Barycentric coordinates:
  - $\mathbf{p} = \alpha\mathbf{a} + \beta\mathbf{b} + \gamma\mathbf{c}$ with $\alpha + \beta + \gamma = 1$
  - Unique for non-collinear **a**,**b**,**c**
  - Ratio of *signed* triangle areas

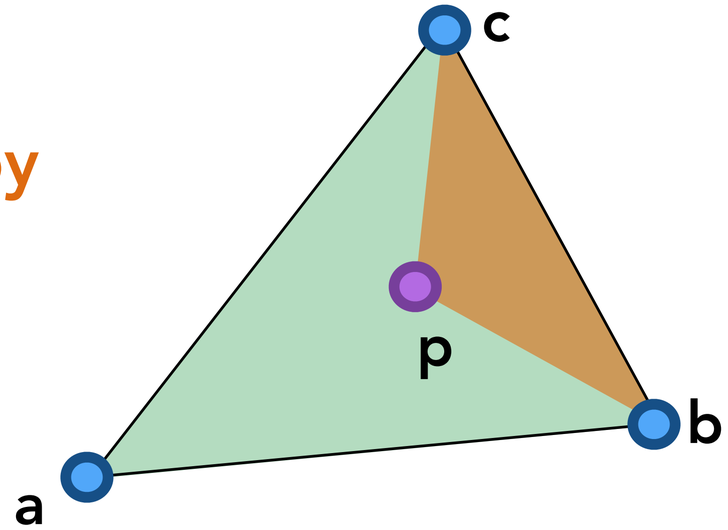
$$\alpha(\mathbf{p}) = \frac{\text{area}(\mathbf{p},\mathbf{b},\mathbf{c})}{\text{area}(\mathbf{a},\mathbf{b},\mathbf{c})}$$

$$\beta(\mathbf{p}) = \frac{\text{area}(\mathbf{p},\mathbf{c},\mathbf{a})}{\text{area}(\mathbf{a},\mathbf{b},\mathbf{c})}$$

$$\gamma(\mathbf{p}) = \frac{\text{area}(\mathbf{p},\mathbf{a},\mathbf{b})}{\text{area}(\mathbf{a},\mathbf{b},\mathbf{c})}$$

The triangle formed by the opposite side

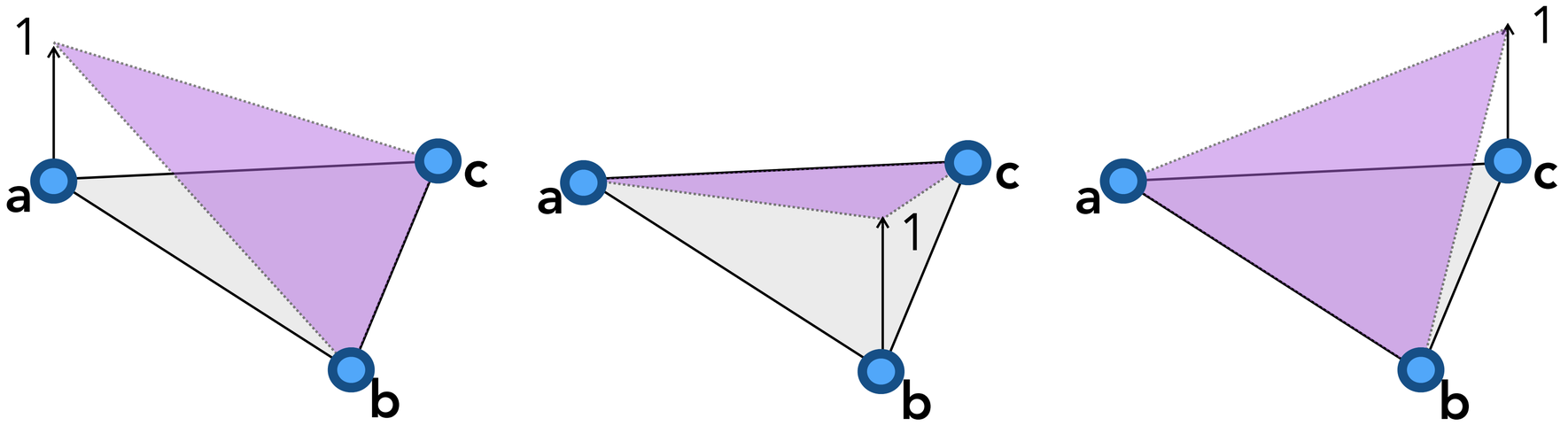
The total area



University of Victoria | Computer Science

Barycentric Interpolation

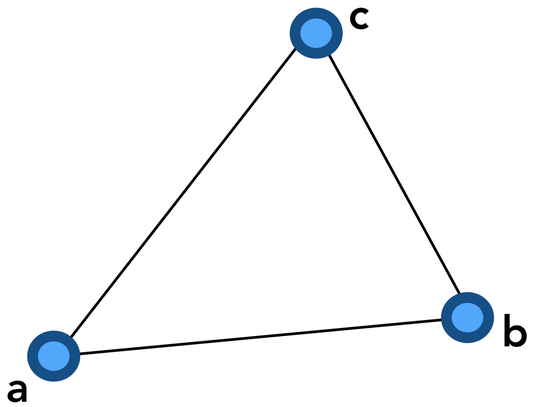
- Barycentric coordinates:
  - $\mathbf{p} = \alpha\mathbf{a} + \beta\mathbf{b} + \gamma\mathbf{c}$ with $\alpha + \beta + \gamma = 1$
  - Unique for non-collinear **a**,**b**,**c**
  - Ratio of triangle areas
  - $\alpha(\mathbf{p})$, $\beta(\mathbf{p})$, $\gamma(\mathbf{p})$ are linear functions



University of Victoria | Computer Science

Barycentric Interpolation

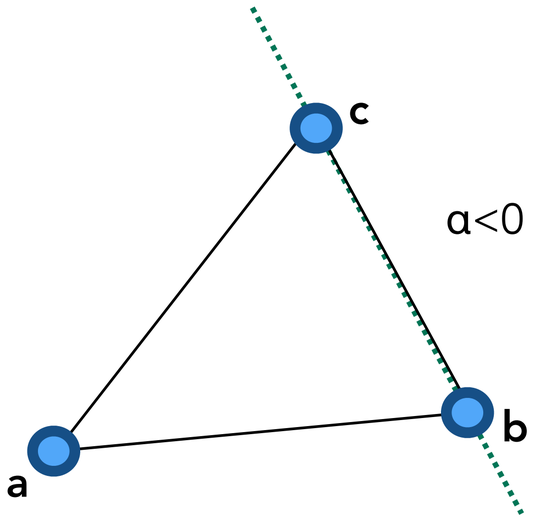
- Barycentric coordinates:
  - $\mathbf{p} = \alpha\mathbf{a} + \beta\mathbf{b} + \gamma\mathbf{c}$ with $\alpha + \beta + \gamma = 1$
  - Unique for non-collinear **a**,**b**,**c**
  - Ratio of triangle areas
  - $\alpha(\mathbf{p})$, $\beta(\mathbf{p})$, $\gamma(\mathbf{p})$ are linear functions
  - Gives inside/outside information



University of Victoria | Computer Science

Barycentric Interpolation

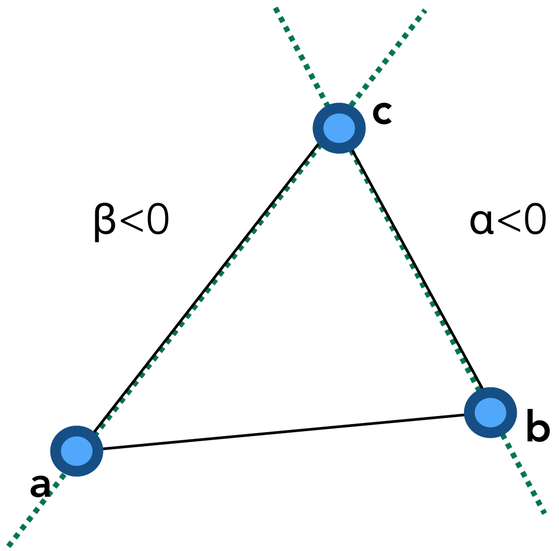
- Barycentric coordinates:
  - $\mathbf{p} = \alpha\mathbf{a} + \beta\mathbf{b} + \gamma\mathbf{c}$ with $\alpha + \beta + \gamma = 1$
  - Unique for non-collinear **a**,**b**,**c**
  - Ratio of triangle areas
  - $\alpha(\mathbf{p})$, $\beta(\mathbf{p})$, $\gamma(\mathbf{p})$ are linear functions
  - Gives inside/outside information



University of Victoria | Computer Science

Barycentric Interpolation

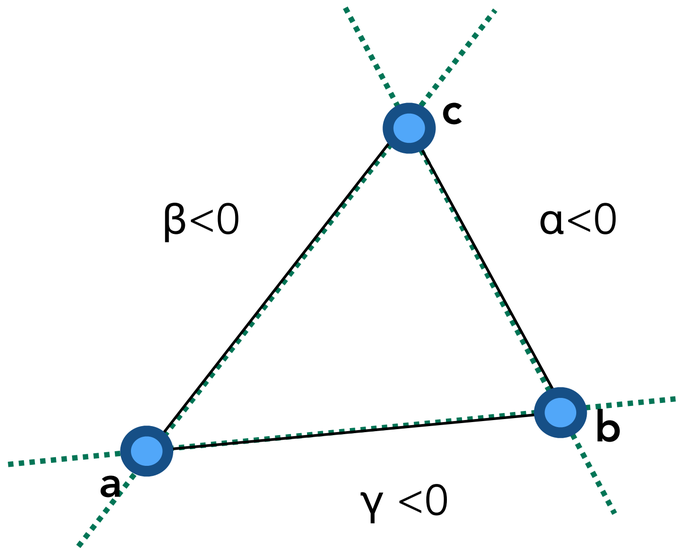
- Barycentric coordinates:
  - $\mathbf{p} = \alpha\mathbf{a} + \beta\mathbf{b} + \gamma\mathbf{c}$ with $\alpha + \beta + \gamma = 1$
  - Unique for non-collinear **a**,**b**,**c**
  - Ratio of triangle areas
  - $\alpha(\mathbf{p})$, $\beta(\mathbf{p})$, $\gamma(\mathbf{p})$ are linear functions
  - Gives inside/outside information



University of Victoria | Computer Science

Barycentric Interpolation

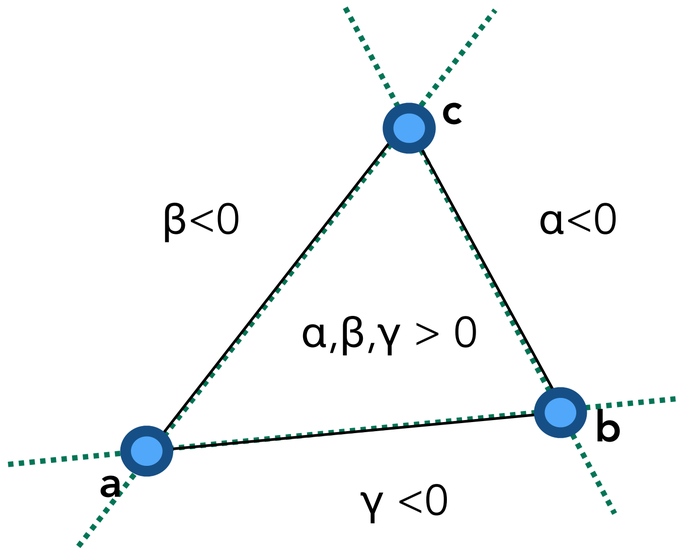
- Barycentric coordinates:
  - $\mathbf{p} = \alpha\mathbf{a} + \beta\mathbf{b} + \gamma\mathbf{c}$ with $\alpha + \beta + \gamma = 1$
  - Unique for non-collinear **a**,**b**,**c**
  - Ratio of triangle areas
  - $\alpha(\mathbf{p})$, $\beta(\mathbf{p})$, $\gamma(\mathbf{p})$ are linear functions
  - Gives inside/outside information



University of Victoria | Computer Science

Barycentric Interpolation

- Barycentric coordinates:
  - $\mathbf{p} = \alpha\mathbf{a} + \beta\mathbf{b} + \gamma\mathbf{c}$ with $\alpha + \beta + \gamma = 1$
  - Unique for non-collinear **a**,**b**,**c**
  - Ratio of triangle areas
  - $\alpha(\mathbf{p})$, $\beta(\mathbf{p})$, $\gamma(\mathbf{p})$ are linear functions
  - Gives inside/outside information

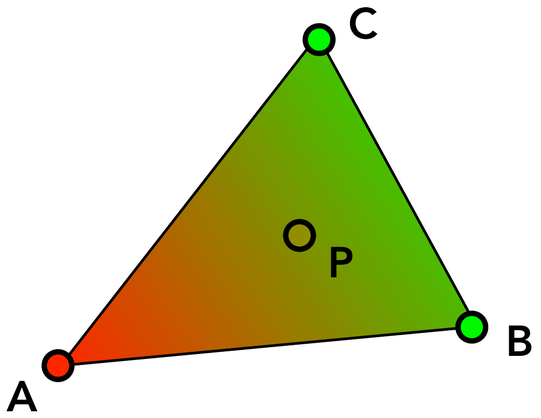


University of Victoria | Computer Science

Barycentric Interpolation

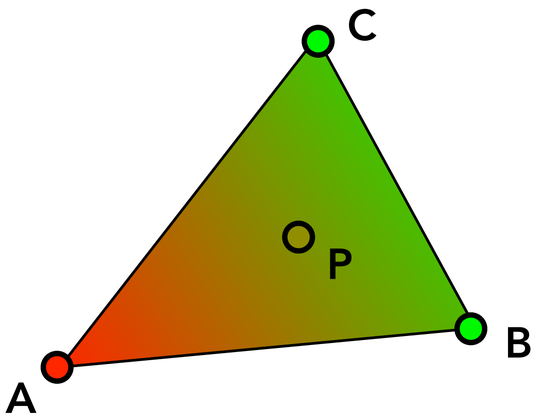
- Barycentric coordinates:
  - **p** = α**a** + β**b** + γ**c** with α + β + γ = 1
  - Unique for non-collinear **a**,**b**,**c**
  - Ratio of triangle areas
  - α(**p**), β(**p**), γ(**p**) are linear functions
  - Gives inside/outside information
  - Use barycentric coordinates to interpolate vertex normals (or other data, e.g. colors)

$$\mathbf{n}(\mathbf{P}) \quad = \quad \alpha \cdot \mathbf{n}(\mathbf{A}) + \beta \cdot \mathbf{n}(\mathbf{B}) + \gamma \cdot \mathbf{n}(\mathbf{C})$$

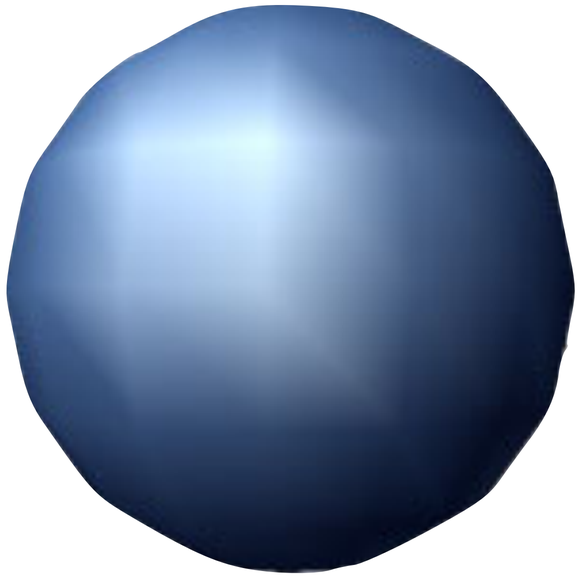


University of Victoria | Computer Science

Color Interpolation

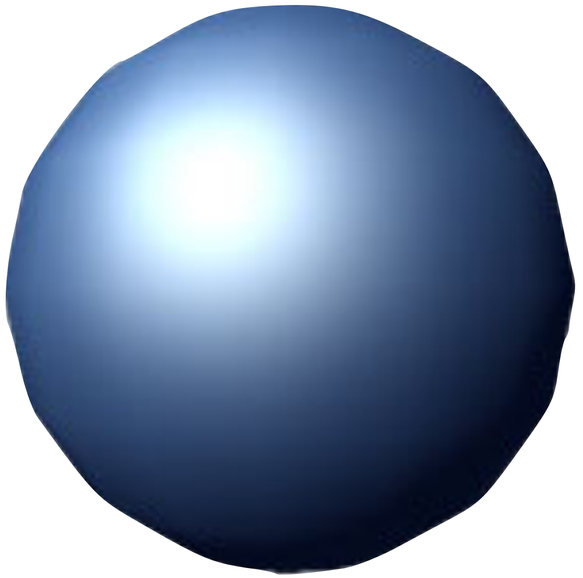


Per-vertex



Evaluate color on vertices, then interpolates it

Per-pixel



Interpolates positions and normals, then evaluate color on each pixel

http://www.cgchannel.com/2010/11/cg-science-for-artists-part-2-the-real-time-rendering-pipeline/

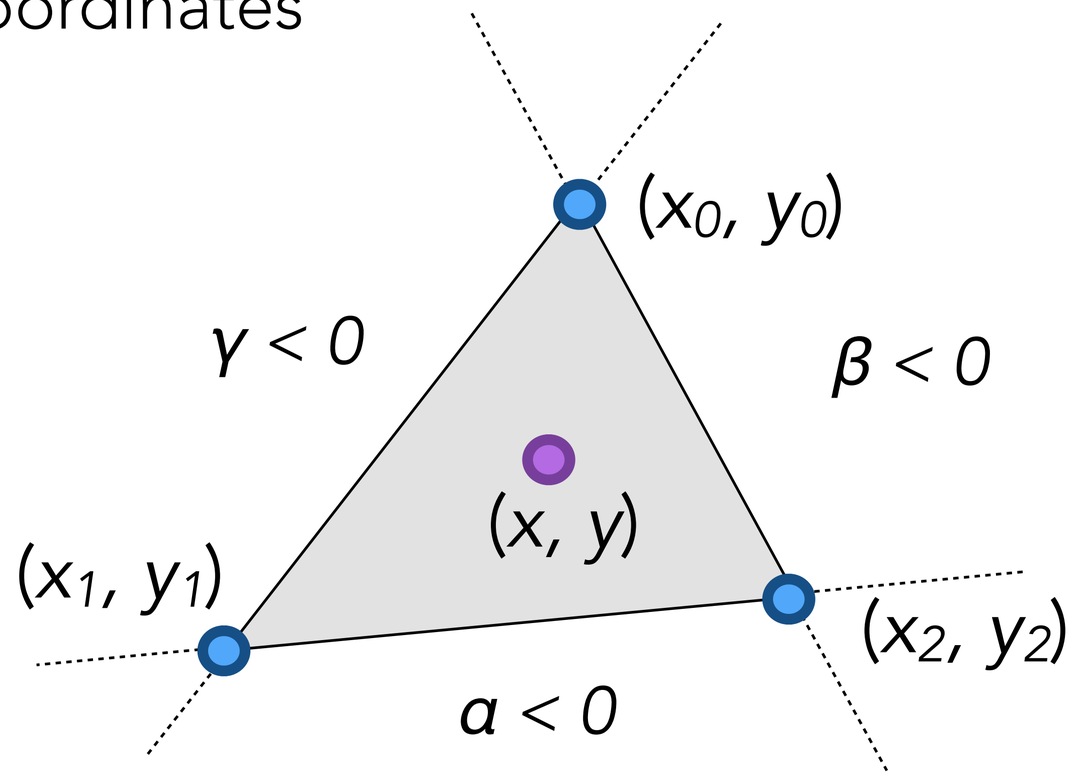
Triangle Rasterization

- Each triangle is represented as three 2D points (x_0, y_0), (x_1, y_1), (x_2, y_2)
- Rasterization using barycentric coordinates

$$x = \alpha \cdot x_0 + \beta \cdot x_1 + \gamma \cdot x_2$$

$$y = \alpha \cdot y_0 + \beta \cdot y_1 + \gamma \cdot y_2$$

$$\alpha + \beta + \gamma = 1$$



University of Victoria | Computer Science

Triangle Rasterization

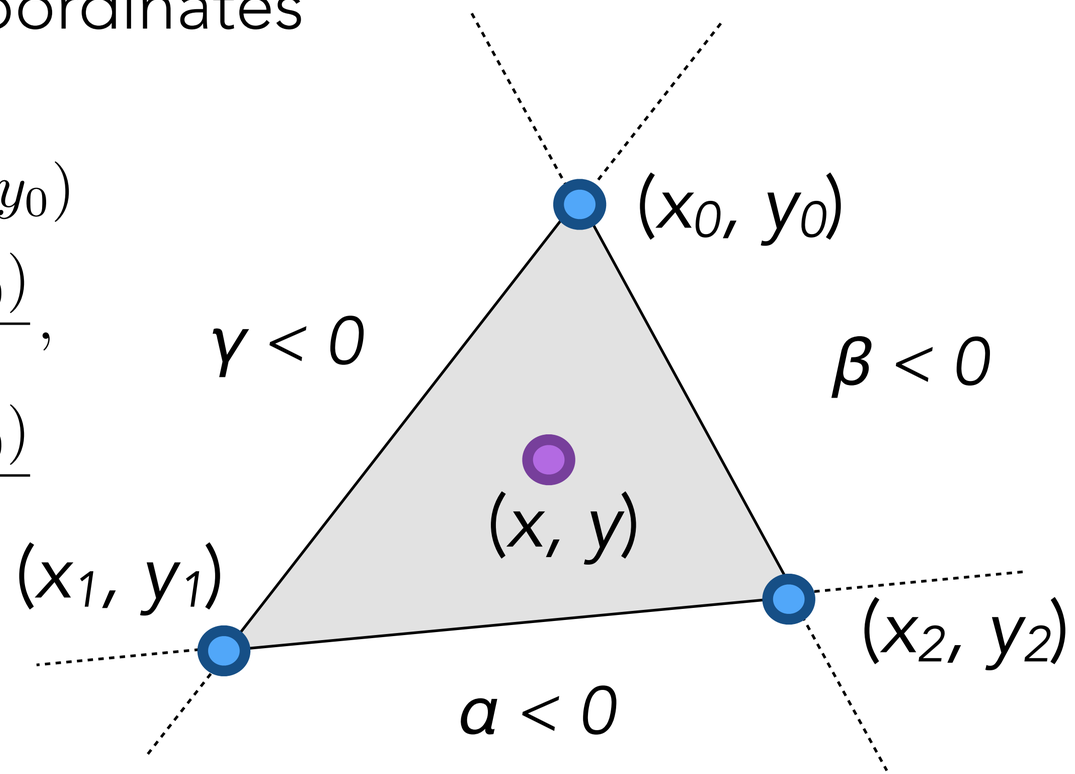
- Each triangle is represented as three 2D points (x_0, y_0), (x_1, y_1), (x_2, y_2)
- Rasterization using barycentric coordinates

$$D = (x_1 - x_0)(y_2 - y_0) - (x_2 - x_0)(y_1 - y_0)$$

$$\beta = \frac{(x - x_0)(y_2 - y_0) - (x_2 - x_0)(y - y_0)}{D},$$

$$\gamma = \frac{(x_1 - x_0)(y - y_0) - (x - x_0)(y_1 - y_0)}{D}$$

$$\alpha = 1 - \beta - \gamma$$



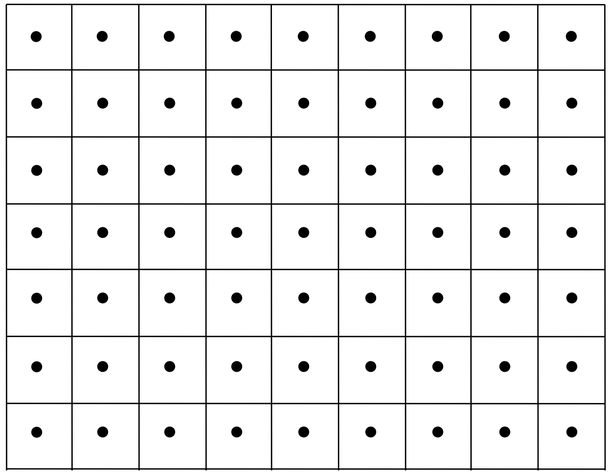
University of Victoria | Computer Science

Triangle Rasterization

- Each triangle is represented as three 2D points

  (x_0, y_0), (x_1, y_1), (x_2, y_2)

- Rasterization using barycentric coordinates



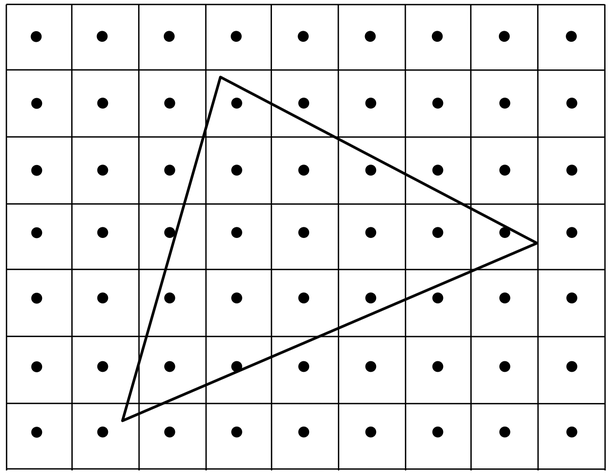
University of Victoria | Computer Science

Triangle Rasterization

- Each triangle is represented as three 2D points

  (x_0, y_0), (x_1, y_1), (x_2, y_2)

- Rasterization using barycentric coordinates

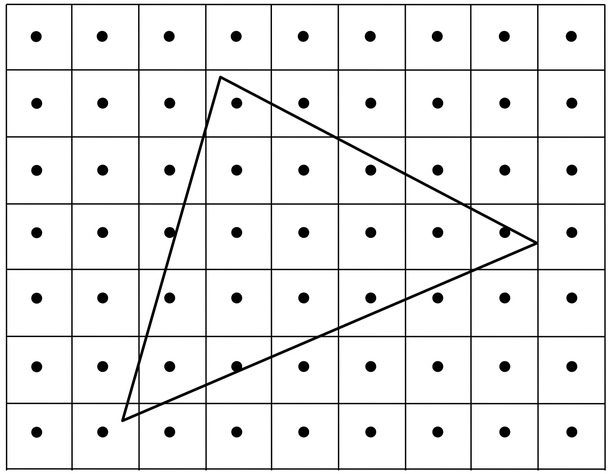


University of Victoria | Computer Science

Triangle Rasterization

- Each triangle is represented as three 2D points
  (x_0, y_0), (x_1, y_1), (x_2, y_2)
- Rasterization using barycentric coordinates

```
for all y do
    for all x do
        compute (α,β,γ) for (x,y)
        if (α ∈ [0,1] and β ∈ [0,1] and γ ∈ [0,1]
            set_pixel (x,y)
```

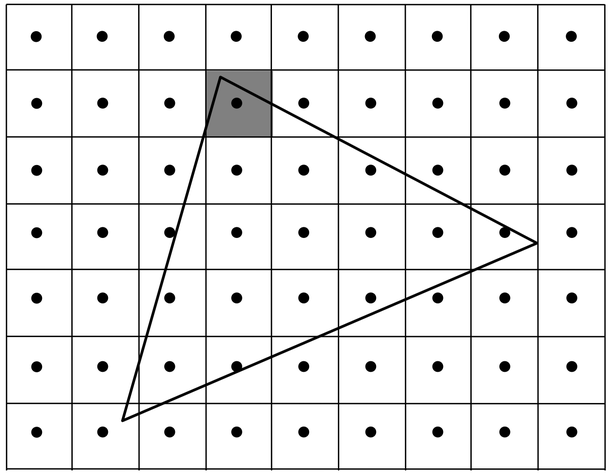


University of Victoria | Computer Science

Triangle Rasterization

- Each triangle is represented as three 2D points
  (x_0, y_0), (x_1, y_1), (x_2, y_2)
- Rasterization using barycentric coordinates

```
for all y do
    for all x do
        compute (α,β,γ) for (x,y)
        if (α ∈ [0,1] and β ∈ [0,1] and γ ∈ [0,1]
            set_pixel (x,y)
```

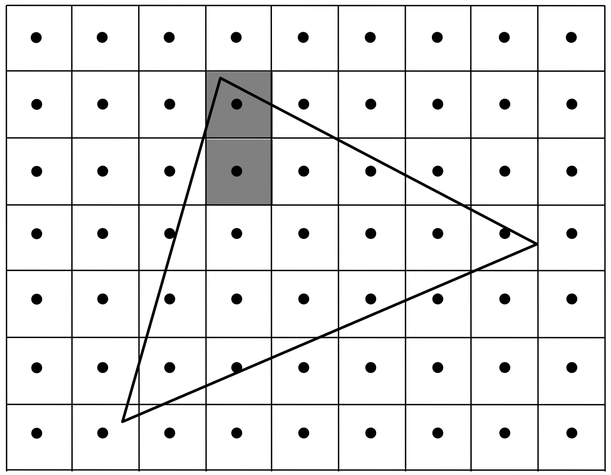


University of Victoria | Computer Science

Triangle Rasterization

- Each triangle is represented as three 2D points
  (x_0, y_0), (x_1, y_1), (x_2, y_2)
- Rasterization using barycentric coordinates

```
for all y do
    for all x do
        compute (α,β,γ) for (x,y)
        if (α ∈ [0,1] and β ∈ [0,1] and γ ∈ [0,1]
            set_pixel (x,y)
```

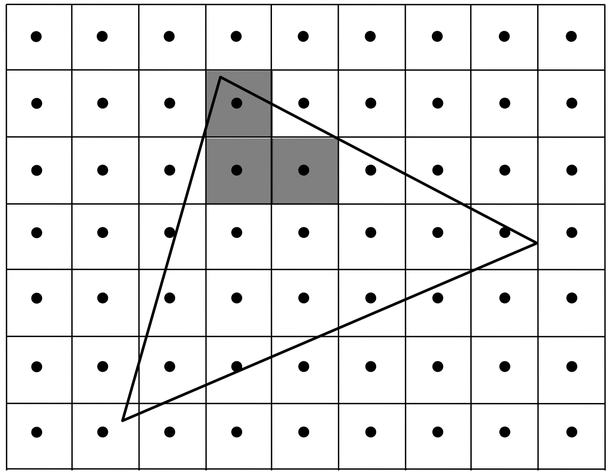


University of Victoria | Computer Science

Triangle Rasterization

- Each triangle is represented as three 2D points
  (x_0, y_0), (x_1, y_1), (x_2, y_2)
- Rasterization using barycentric coordinates

```
for all y do
    for all x do
        compute (α,β,γ) for (x,y)
        if (α ∈ [0,1] and β ∈ [0,1] and γ ∈ [0,1]
            set_pixel (x,y)
```

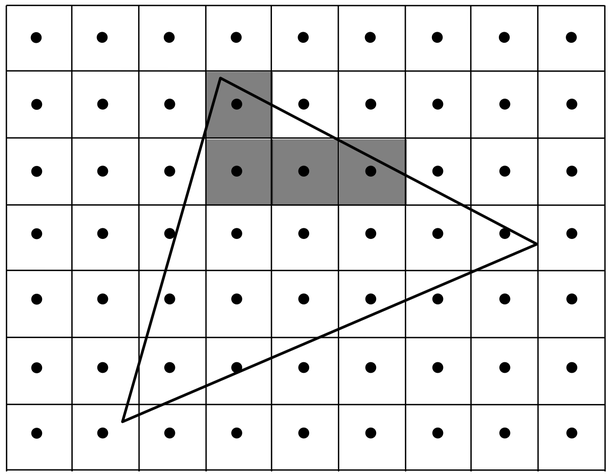


University of Victoria | Computer Science

Triangle Rasterization

- Each triangle is represented as three 2D points
  (x_0, y_0), (x_1, y_1), (x_2, y_2)
- Rasterization using barycentric coordinates

```
for all y do
    for all x do
        compute (α,β,γ) for (x,y)
        if (α ∈ [0,1] and β ∈ [0,1] and γ ∈ [0,1]
            set_pixel (x,y)
```

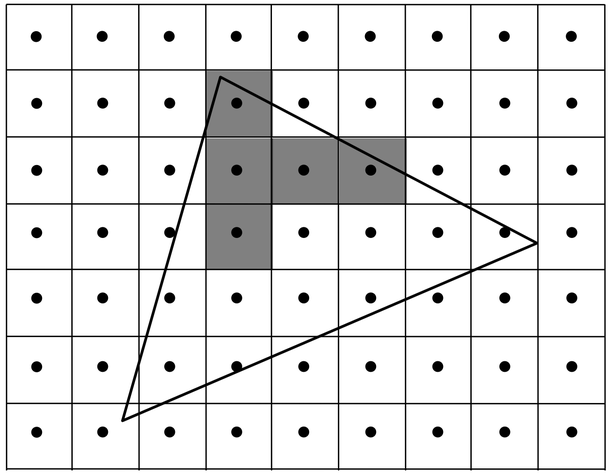


University of Victoria | Computer Science

Triangle Rasterization

- Each triangle is represented as three 2D points
  (x_0, y_0), (x_1, y_1), (x_2, y_2)
- Rasterization using barycentric coordinates

```
for all y do
    for all x do
        compute (α,β,γ) for (x,y)
        if (α ∈ [0,1] and β ∈ [0,1] and γ ∈ [0,1]
            set_pixel (x,y)
```

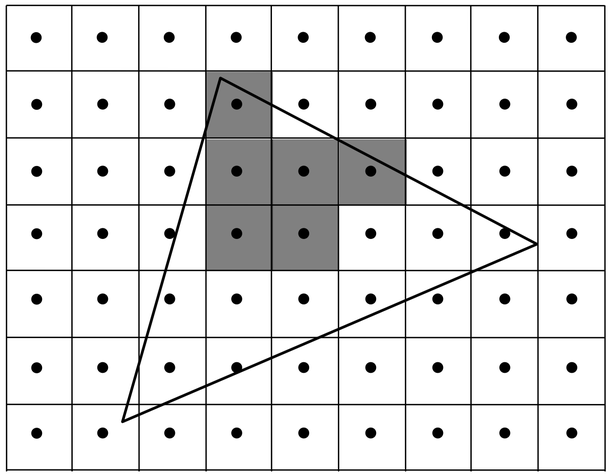


University of Victoria | Computer Science

Triangle Rasterization

- Each triangle is represented as three 2D points
  (x_0, y_0), (x_1, y_1), (x_2, y_2)
- Rasterization using barycentric coordinates

```
for all y do
    for all x do
        compute (α,β,γ) for (x,y)
        if (α ∈ [0,1] and β ∈ [0,1] and γ ∈ [0,1]
            set_pixel (x,y)
```

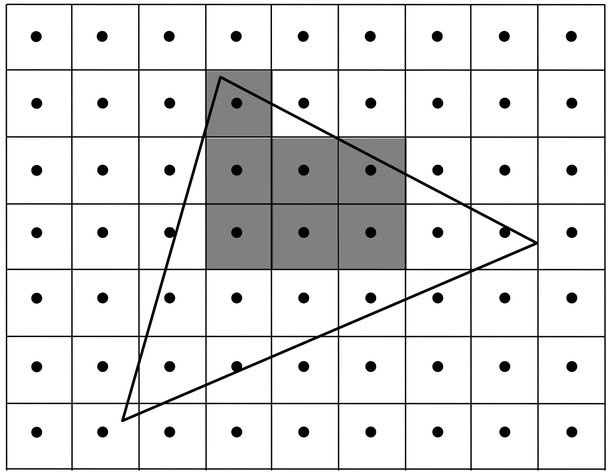


University of Victoria | Computer Science

Triangle Rasterization

- Each triangle is represented as three 2D points
  (x_0, y_0), (x_1, y_1), (x_2, y_2)
- Rasterization using barycentric coordinates

```
for all y do
    for all x do
        compute (α,β,γ) for (x,y)
        if (α ∈ [0,1] and β ∈ [0,1] and γ ∈ [0,1]
            set_pixel (x,y)
```

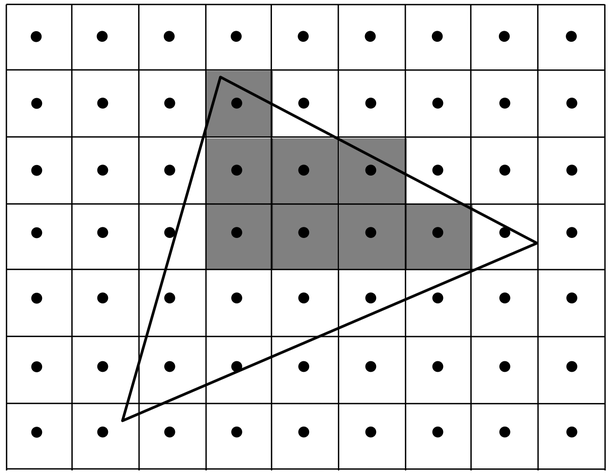


University of Victoria | Computer Science

Triangle Rasterization

- Each triangle is represented as three 2D points
  (x_0, y_0), (x_1, y_1), (x_2, y_2)
- Rasterization using barycentric coordinates

```
for all y do
    for all x do
        compute (α,β,γ) for (x,y)
        if (α ∈ [0,1] and β ∈ [0,1] and γ ∈ [0,1]
            set_pixel (x,y)
```

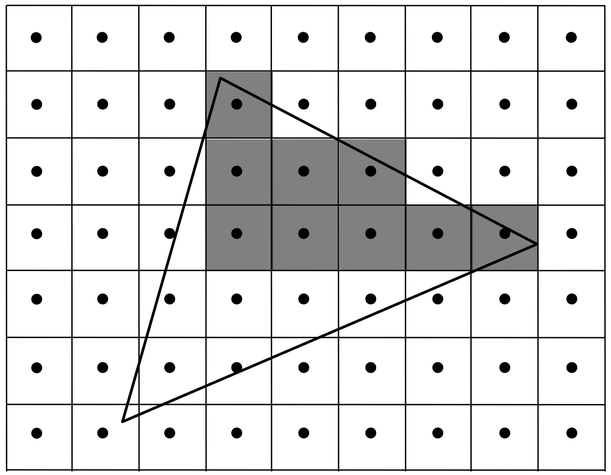


University of Victoria | Computer Science

Triangle Rasterization

- Each triangle is represented as three 2D points
  (x_0, y_0), (x_1, y_1), (x_2, y_2)
- Rasterization using barycentric coordinates

```
for all y do
    for all x do
        compute (α,β,γ) for (x,y)
        if (α ∈ [0,1] and β ∈ [0,1] and γ ∈ [0,1]
            set_pixel (x,y)
```

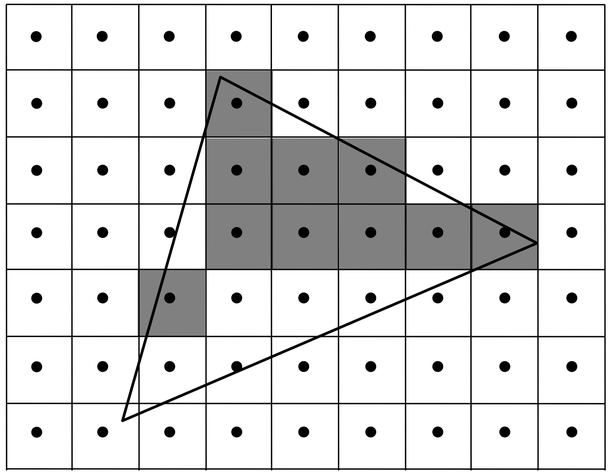


University of Victoria | Computer Science

Triangle Rasterization

- Each triangle is represented as three 2D points
  (x_0, y_0), (x_1, y_1), (x_2, y_2)
- Rasterization using barycentric coordinates

```
for all y do
    for all x do
        compute (α,β,γ) for (x,y)
        if (α ∈ [0,1] and β ∈ [0,1] and γ ∈ [0,1]
            set_pixel (x,y)
```

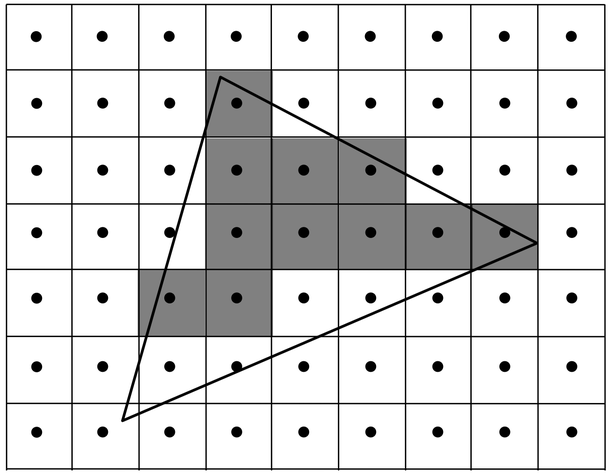


University of Victoria | Computer Science

Triangle Rasterization

- Each triangle is represented as three 2D points
  (x_0, y_0), (x_1, y_1), (x_2, y_2)
- Rasterization using barycentric coordinates

```
for all y do
    for all x do
        compute (α,β,γ) for (x,y)
        if (α ∈ [0,1] and β ∈ [0,1] and γ ∈ [0,1]
            set_pixel (x,y)
```

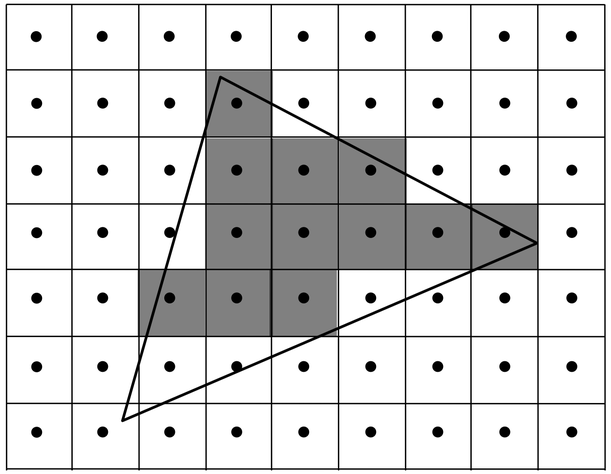


University of Victoria | Computer Science

Triangle Rasterization

- Each triangle is represented as three 2D points
  (x_0, y_0), (x_1, y_1), (x_2, y_2)
- Rasterization using barycentric coordinates

```
for all y do
    for all x do
        compute (α,β,γ) for (x,y)
        if (α ∈ [0,1] and β ∈ [0,1] and γ ∈ [0,1]
            set_pixel (x,y)
```

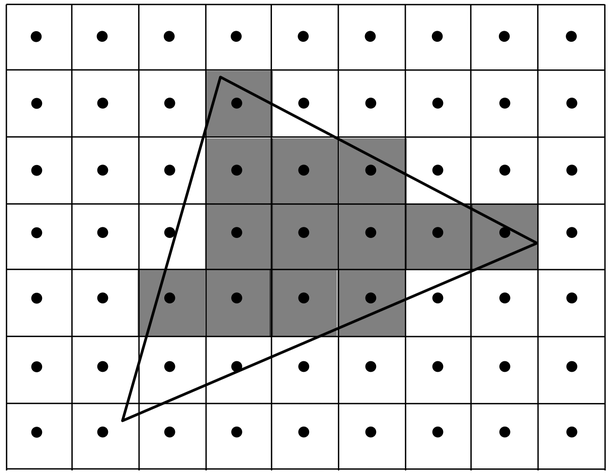


University of Victoria | Computer Science

Triangle Rasterization

- Each triangle is represented as three 2D points
  (x_0, y_0), (x_1, y_1), (x_2, y_2)
- Rasterization using barycentric coordinates

```
for all y do
    for all x do
        compute (α,β,γ) for (x,y)
        if (α ∈ [0,1] and β ∈ [0,1] and γ ∈ [0,1]
            set_pixel (x,y)
```

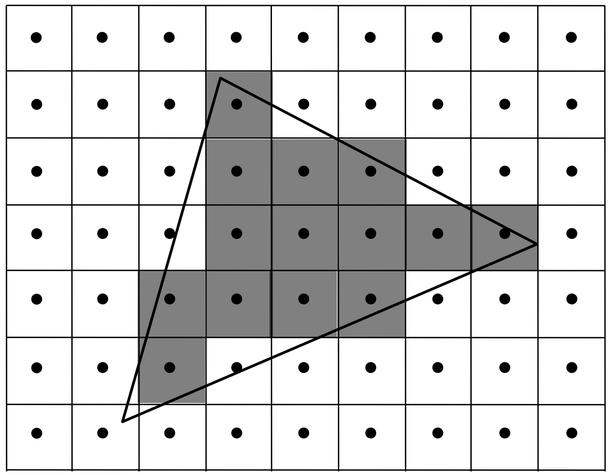


University of Victoria | Computer Science

Triangle Rasterization

- Each triangle is represented as three 2D points
  (x_0, y_0), (x_1, y_1), (x_2, y_2)
- Rasterization using barycentric coordinates

```
for all y do
    for all x do
        compute (α,β,γ) for (x,y)
        if (α ∈ [0,1] and β ∈ [0,1] and γ ∈ [0,1]
            set_pixel (x,y)
```

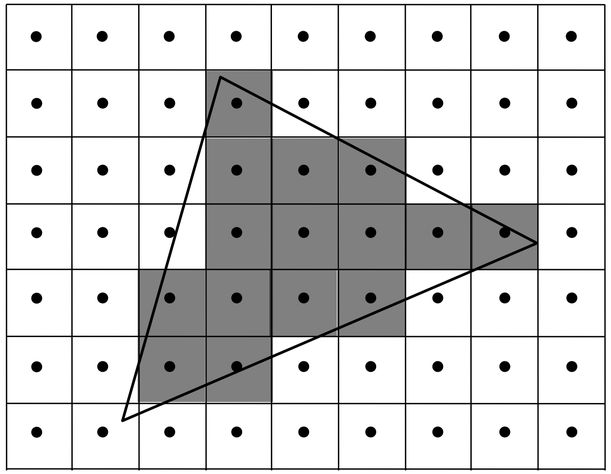


University of Victoria | Computer Science

Triangle Rasterization

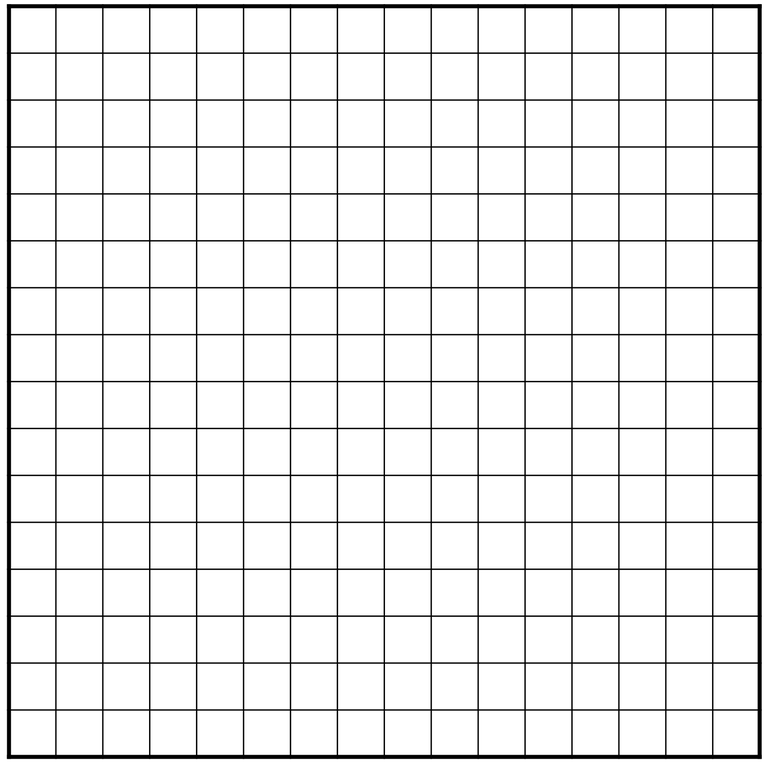
- Each triangle is represented as three 2D points
  (x_0, y_0), (x_1, y_1), (x_2, y_2)
- Rasterization using barycentric coordinates

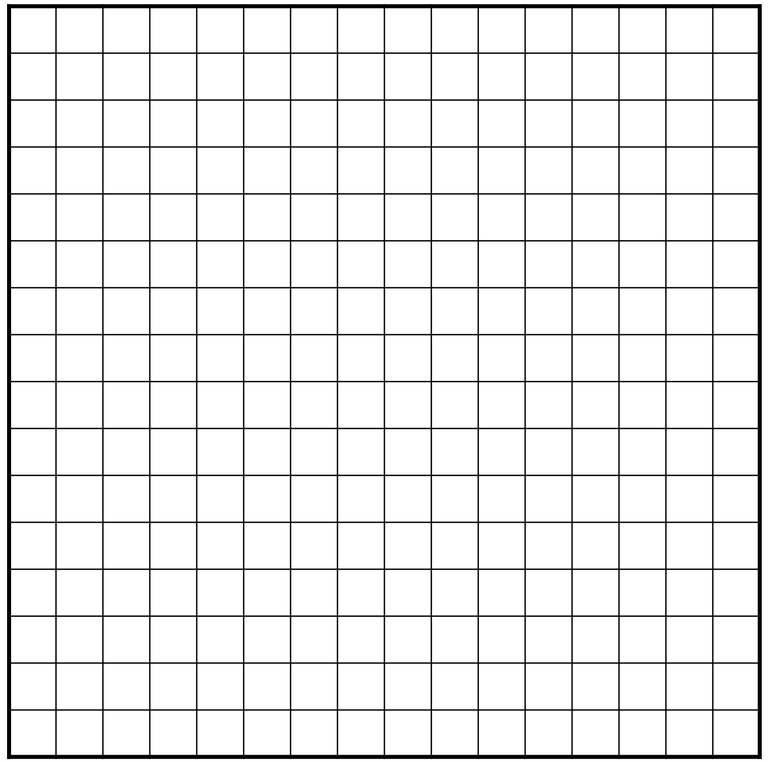
```
for all y do
    for all x do
        compute (α,β,γ) for (x,y)
        if (α ∈ [0,1] and β ∈ [0,1] and γ ∈ [0,1]
            set_pixel (x,y)
```



University of Victoria | Computer Science

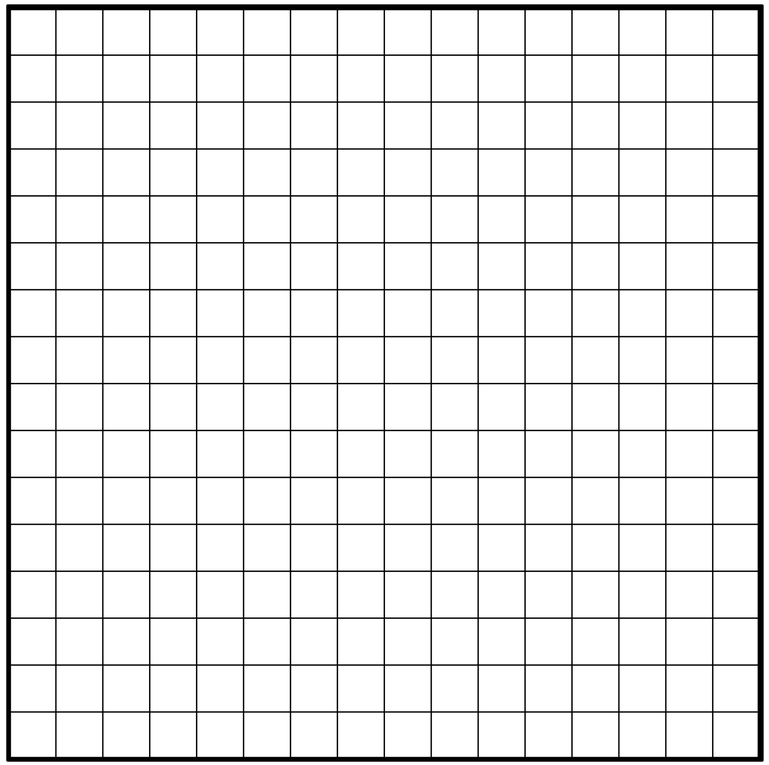
Clipping

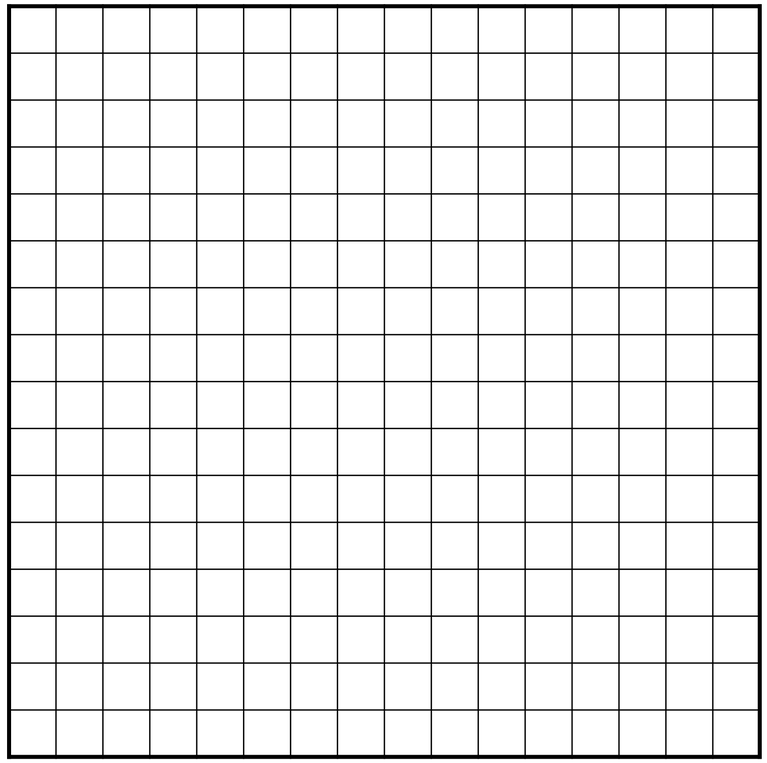




- Ok if you do it brute force
- Care is required if you are explicitly tracing the boundaries

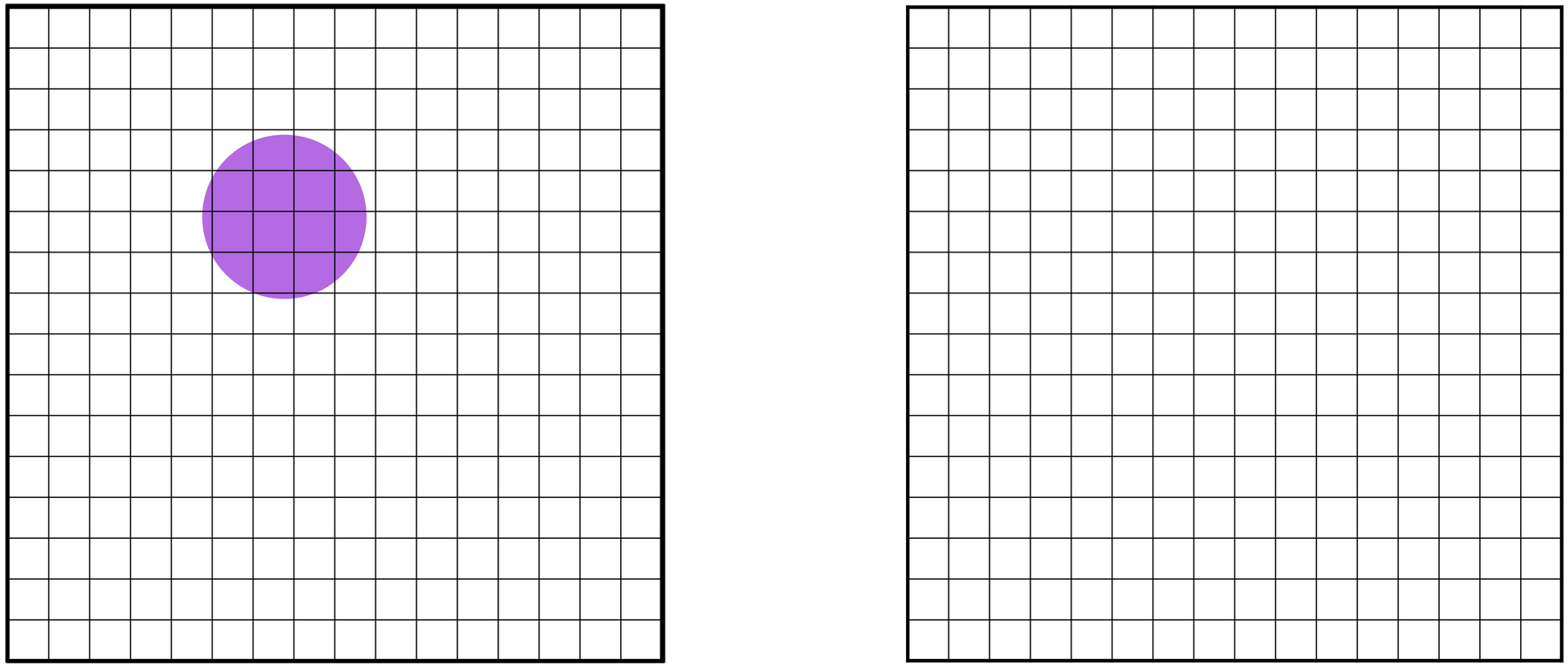
Clipping





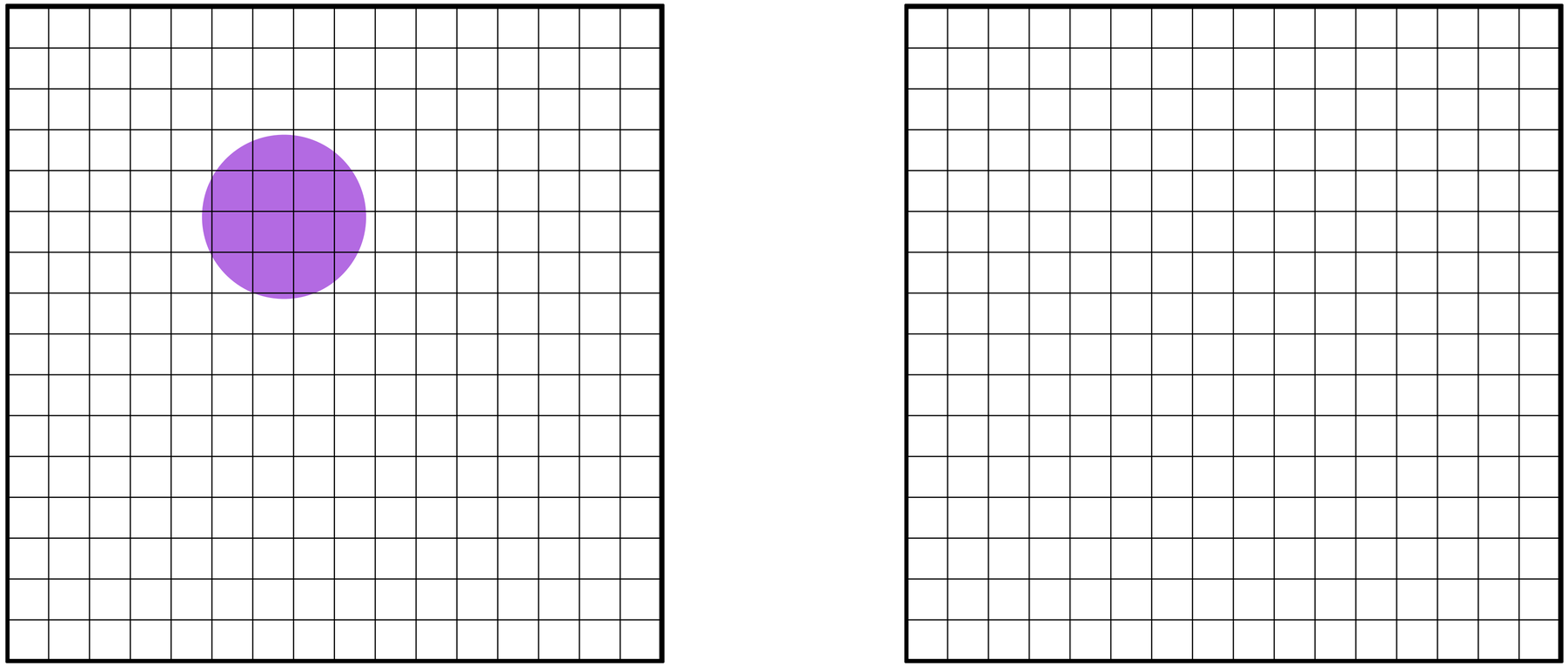
- Ok if you do it brute force
- Care is required if you are explicitly tracing the boundaries

Clipping



- Ok if you do it brute force
- Care is required if you are explicitly tracing the boundaries

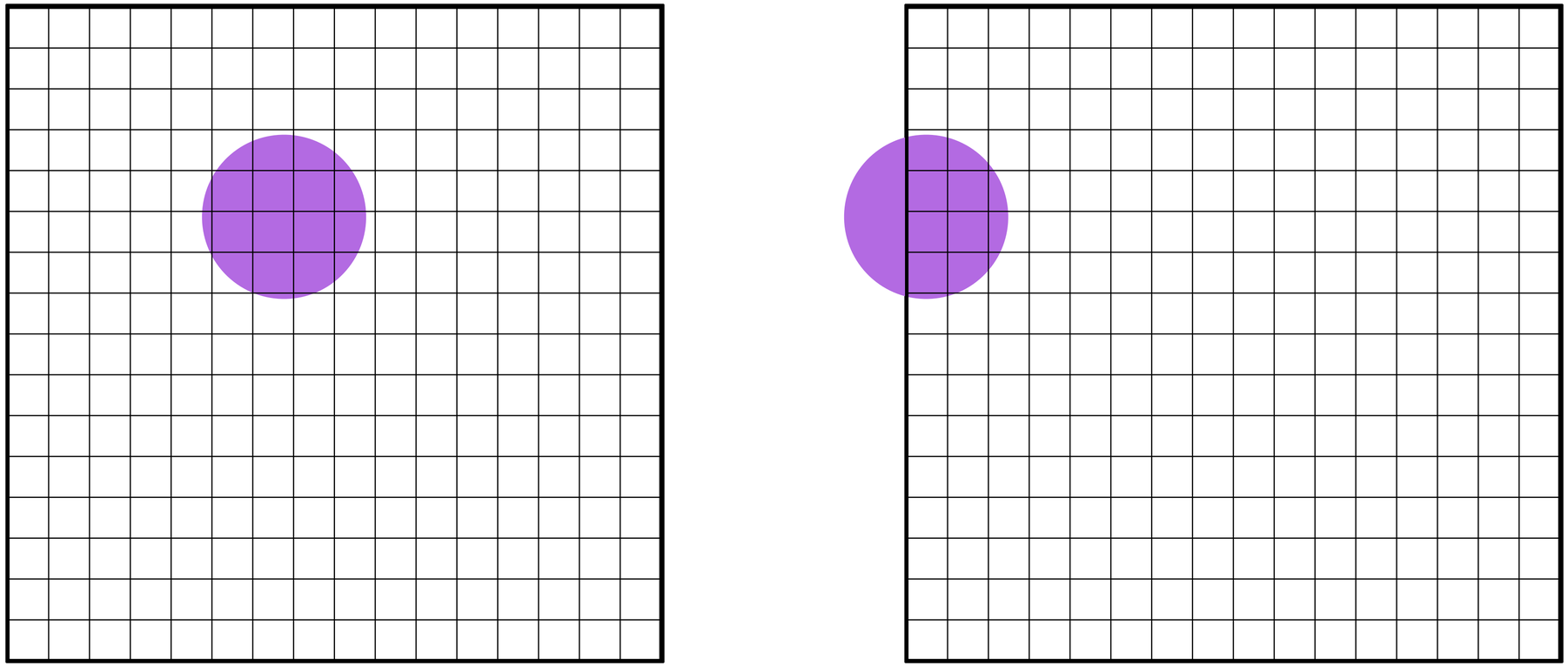
Clipping



- Ok if you do it brute force
- Care is required if you are explicitly tracing the boundaries

University of Victoria | Computer Science

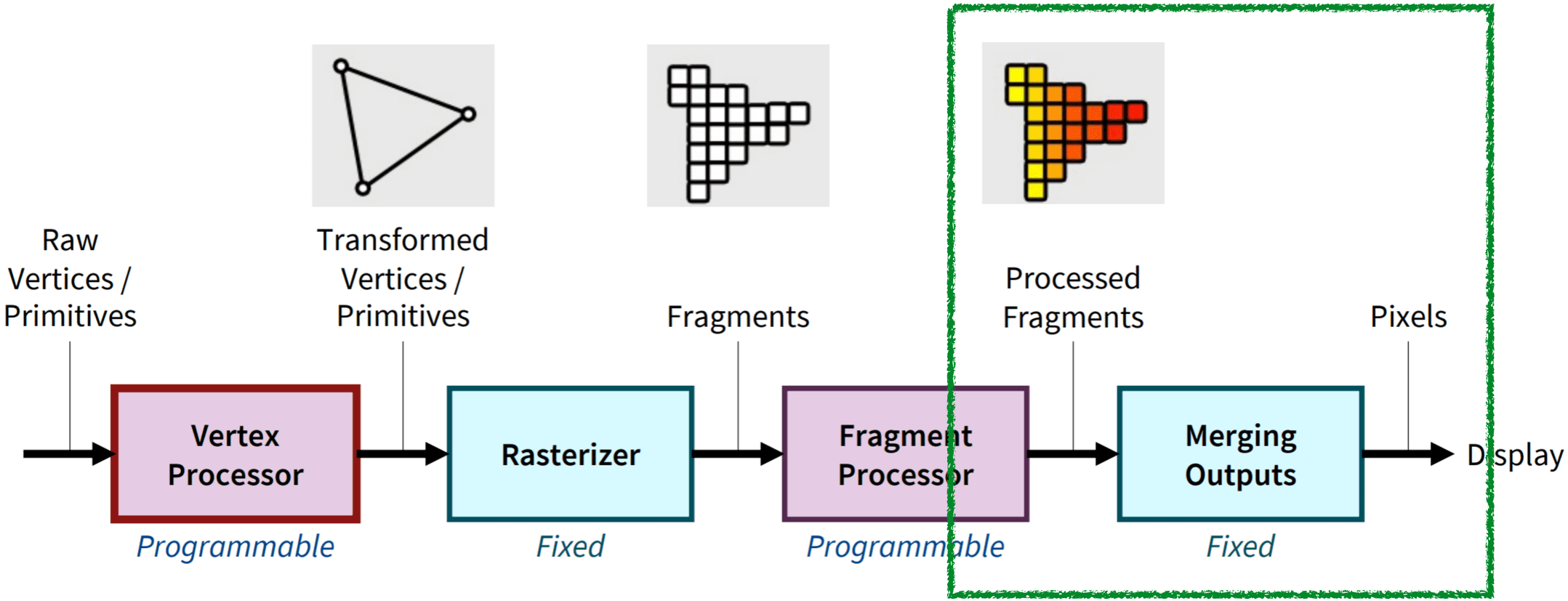
Clipping



- Ok if you do it brute force
- Care is required if you are explicitly tracing the boundaries

University of Victoria | Computer Science

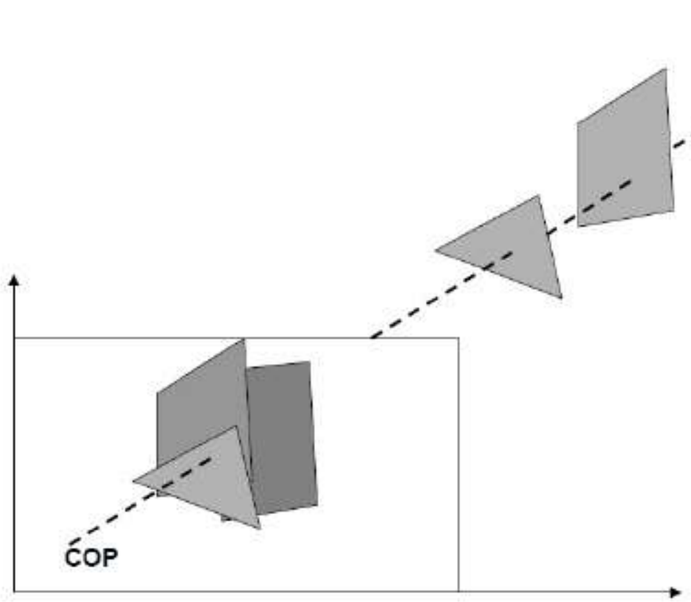
Ray tracing → Rasterization

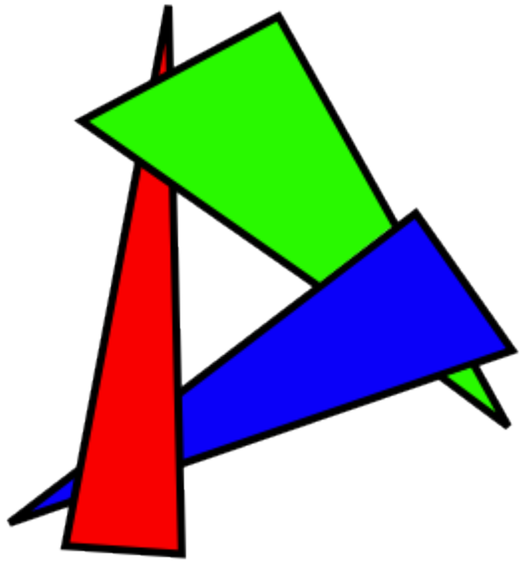


Youngdo Lee; https://leeyngdo.github.io/blog/computer-graphics/2024-02-29-graphics-pipeline/

Objects Depth Sorting

- To handle occlusion, you can sort all the objects in a scene by depth
- This is not always possible!

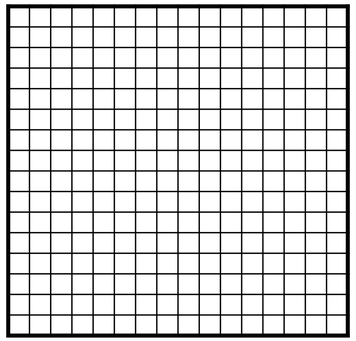




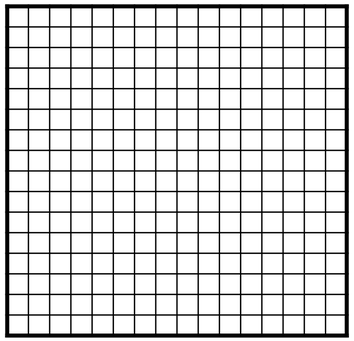
$$\begin{bmatrix} x_{screen} \\ y_{screen} \\ z_{canonical} \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{n_x}{2} & 0 & 0 & \frac{n_x-1}{2} \\ 0 & \frac{n_y}{2} & 0 & \frac{n_y-1}{2} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_{canonical} \\ y_{canonical} \\ z_{canonical} \\ 1 \end{bmatrix}$$

University of Victoria | Computer Science

z-buffering

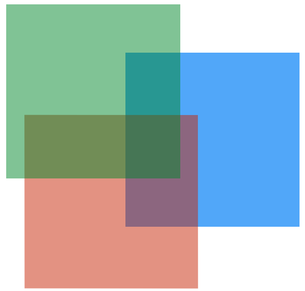


Image



Depth (z)

$$\mathbf{c} = \alpha \mathbf{c_{new}} + (1 - \alpha)\mathbf{c_{old}}$$



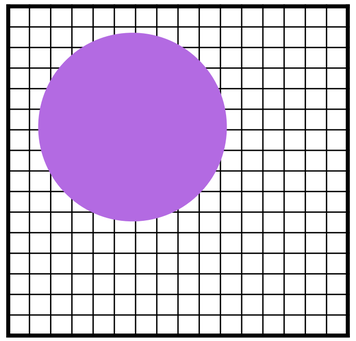


Ordering matters

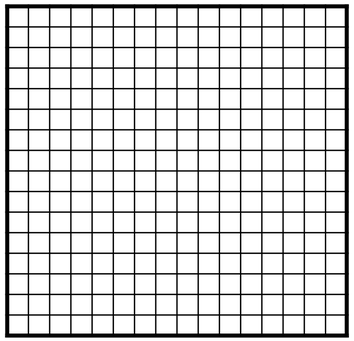
- You render the image both in the Image and in the depth buffer, where you store only the depth
- When a new fragment comes in, you draw it in the image only if it is closer
- This always work and it is cheap to evaluate! It is the default in all graphics hardware
- You still have to sort for transparency…

University of Victoria | Computer Science

z-buffering

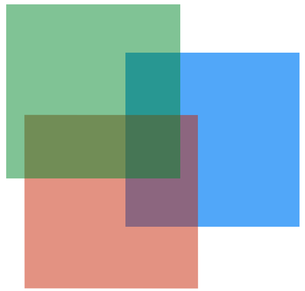


Image



Depth (z)

$$\mathbf{c} = \alpha \mathbf{c_{new}} + (1 - \alpha)\mathbf{c_{old}}$$



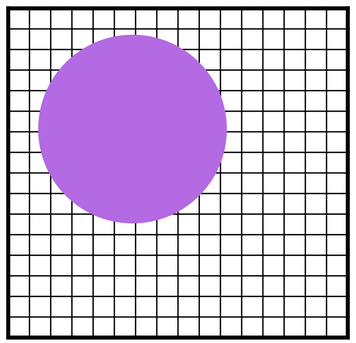


Ordering matters

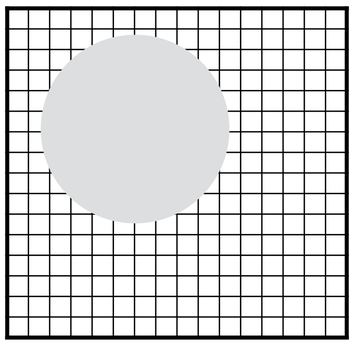
- You render the image both in the Image and in the depth buffer, where you store only the depth
- When a new fragment comes in, you draw it in the image only if it is closer
- This always work and it is cheap to evaluate! It is the default in all graphics hardware
- You still have to sort for transparency…

University of Victoria | Computer Science

z-buffering

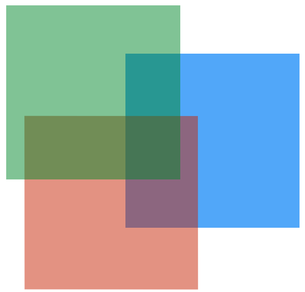


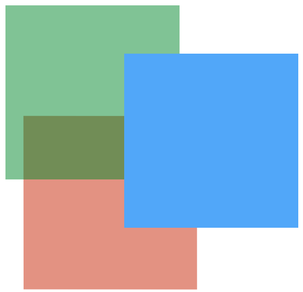
Image



Depth (z)

$$\mathbf{c} = \alpha \mathbf{c_{new}} + (1 - \alpha)\mathbf{c_{old}}$$



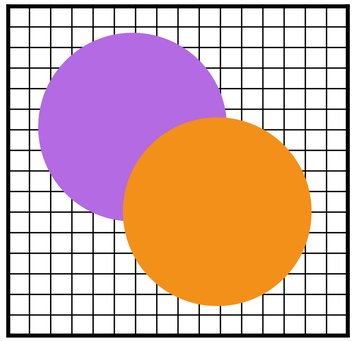


Ordering matters

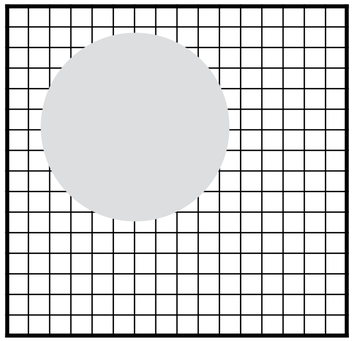
- You render the image both in the Image and in the depth buffer, where you store only the depth
- When a new fragment comes in, you draw it in the image only if it is closer
- This always work and it is cheap to evaluate! It is the default in all graphics hardware
- You still have to sort for transparency…

University of Victoria | Computer Science

z-buffering

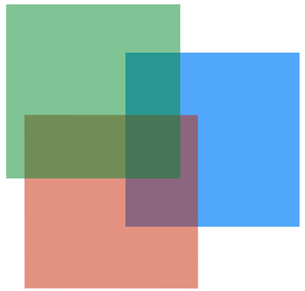


Image



Depth (z)

$$\mathbf{c} = \alpha \mathbf{c_{new}} + (1 - \alpha)\mathbf{c_{old}}$$



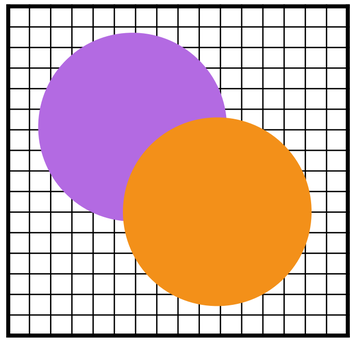


Ordering matters

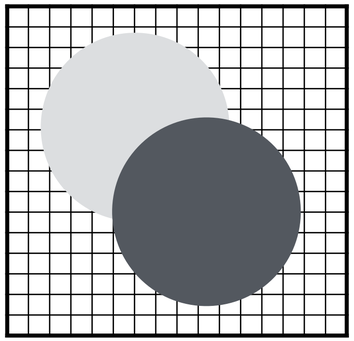
- You render the image both in the Image and in the depth buffer, where you store only the depth
- When a new fragment comes in, you draw it in the image only if it is closer
- This always work and it is cheap to evaluate! It is the default in all graphics hardware
- You still have to sort for transparency…

University of Victoria | Computer Science

z-buffering

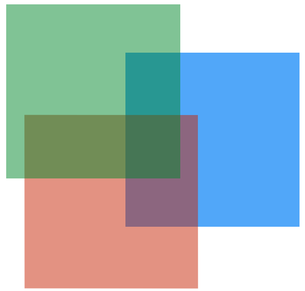


Image



Depth (z)

$$\mathbf{c} = \alpha \mathbf{c_{new}} + (1 - \alpha)\mathbf{c_{old}}$$





Ordering matters

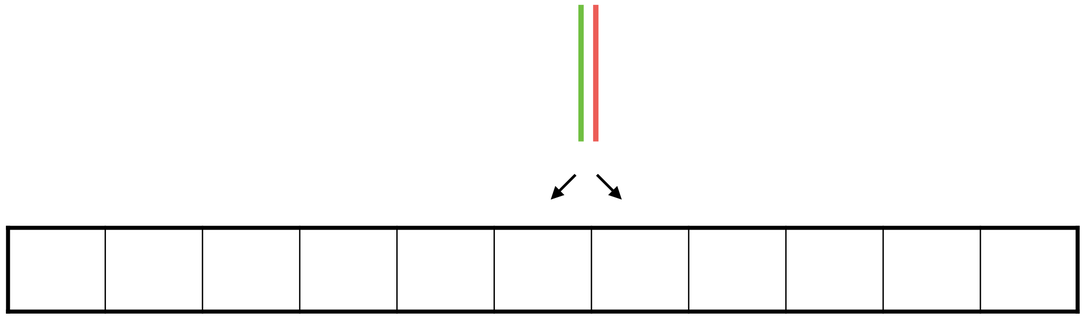
- You render the image both in the Image and in the depth buffer, where you store only the depth
- When a new fragment comes in, you draw it in the image only if it is closer
- This always work and it is cheap to evaluate! It is the default in all graphics hardware
- You still have to sort for transparency…

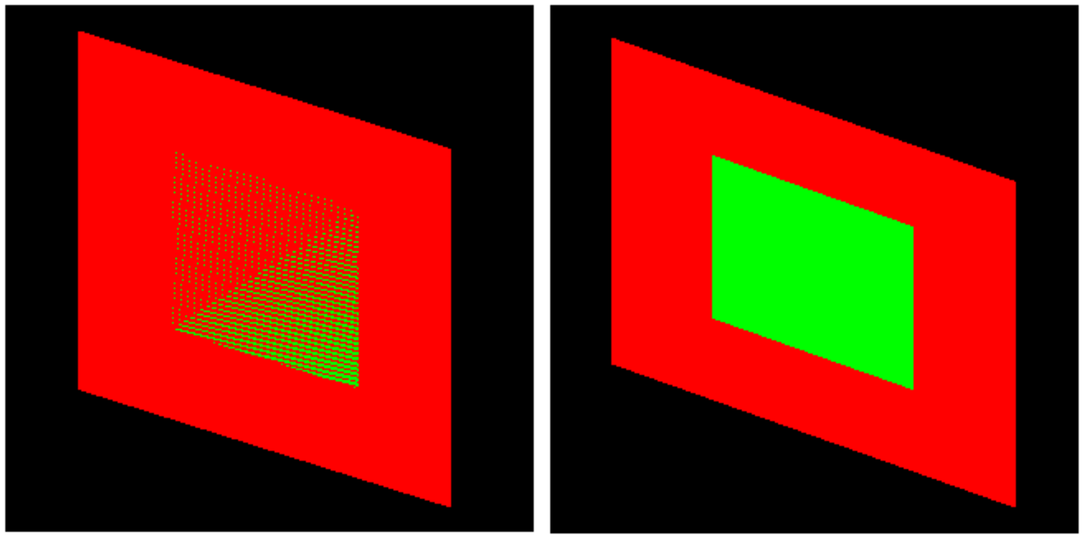
University of Victoria | Computer Science

z-buffer quantization and “z-fighting”

- The z-buffer is quantized (the number of bits is heavily dependent on the hardware platform)

- Two close object might be quantized differently, leading to strange artifacts, usually called “z-fighting”





University of Victoria | Computer Science

Super Sampling Anti-Aliasing





- Render nxn pixels instead of one
- Assign the average to the pixel

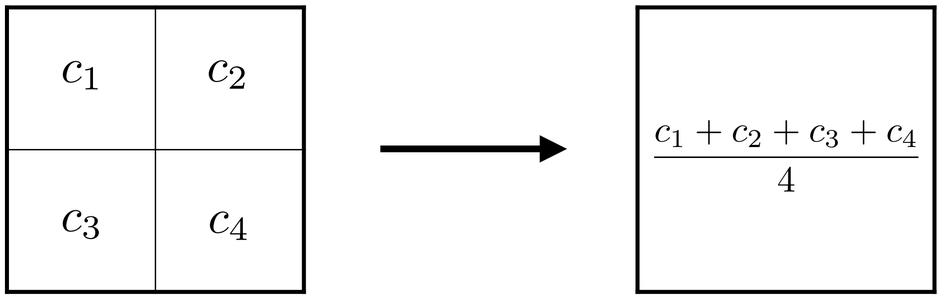
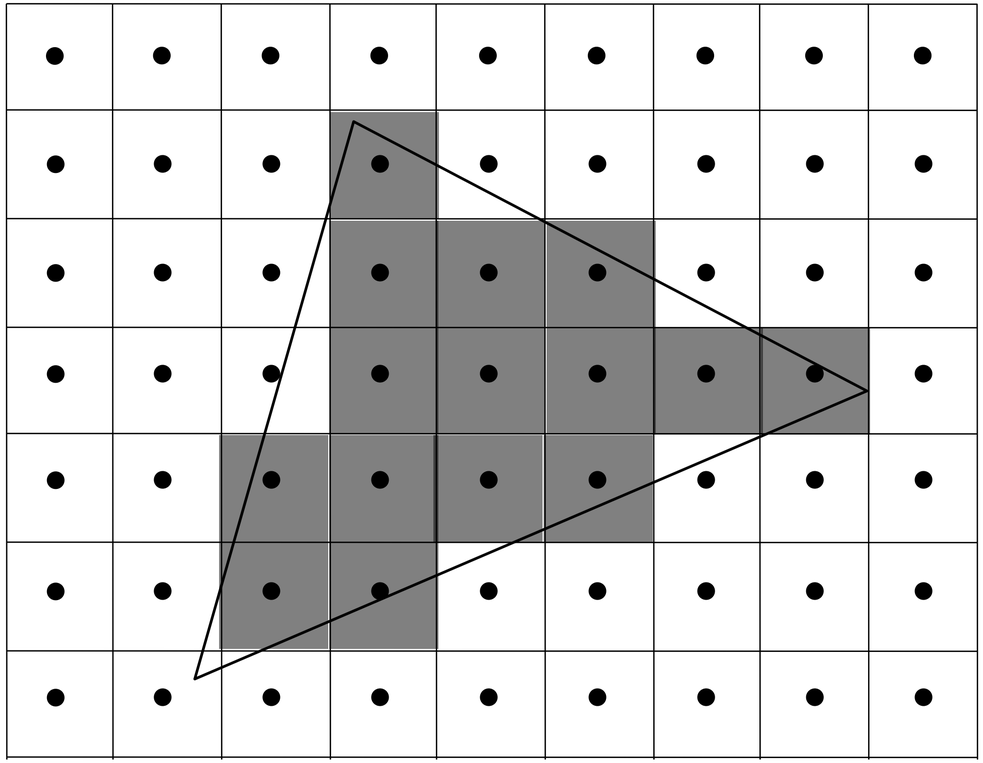


Image Copyright: Fritz Kessler

Super Sampling Anti-Aliasing



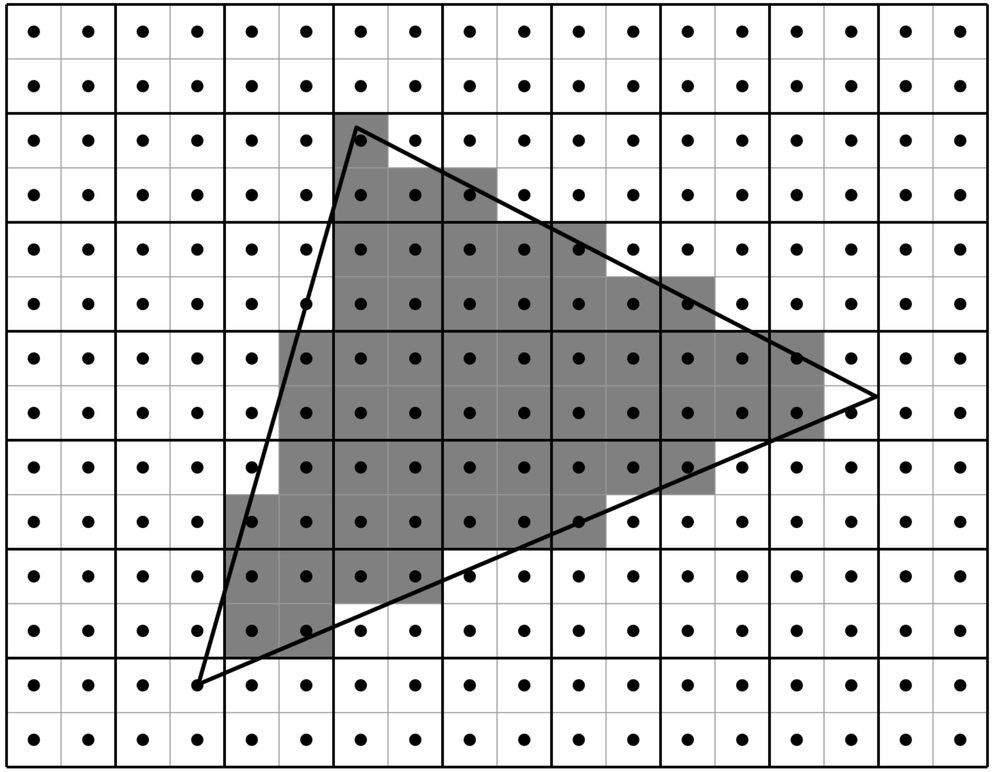
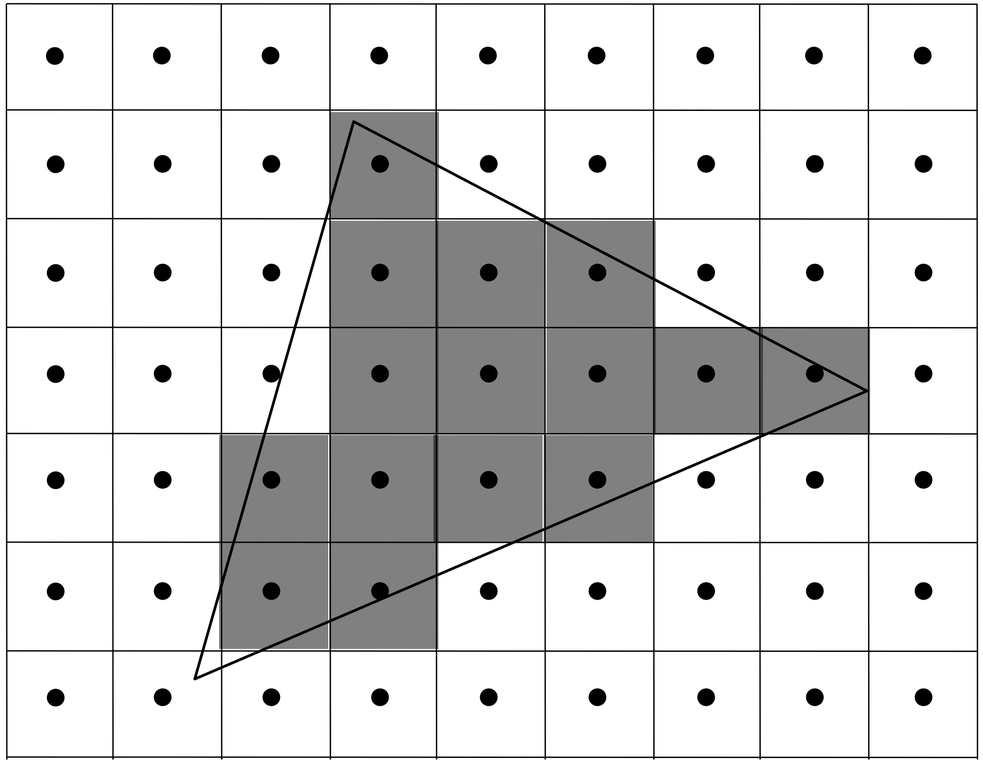


Image Copyright: Fritz Kessler

Super Sampling Anti-Aliasing



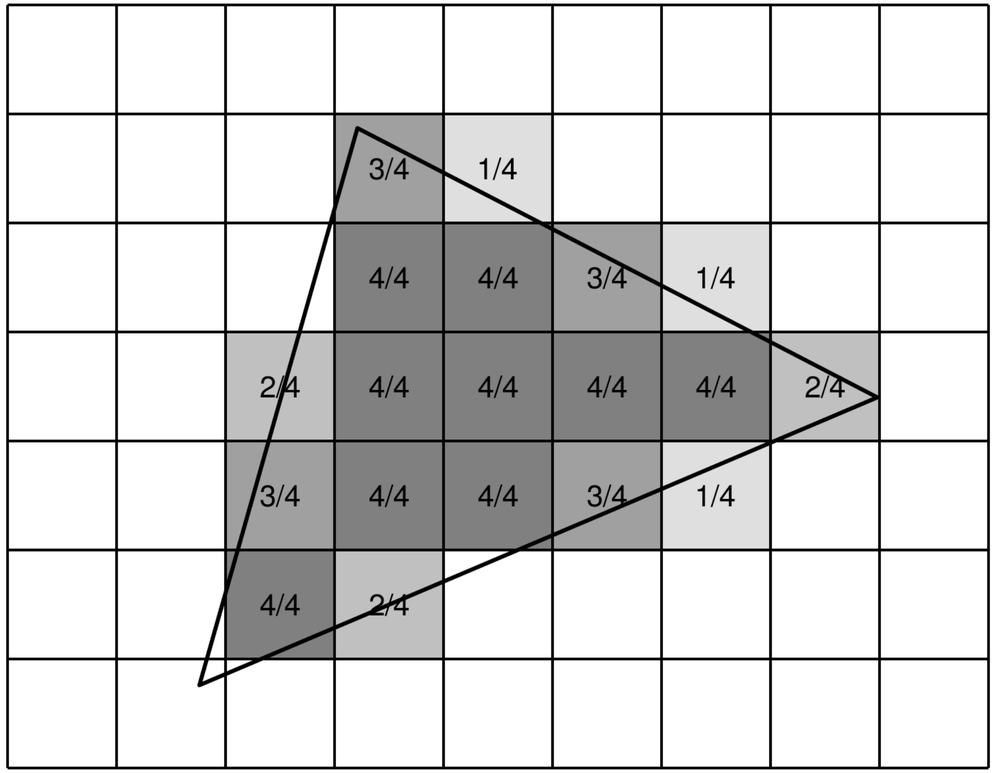
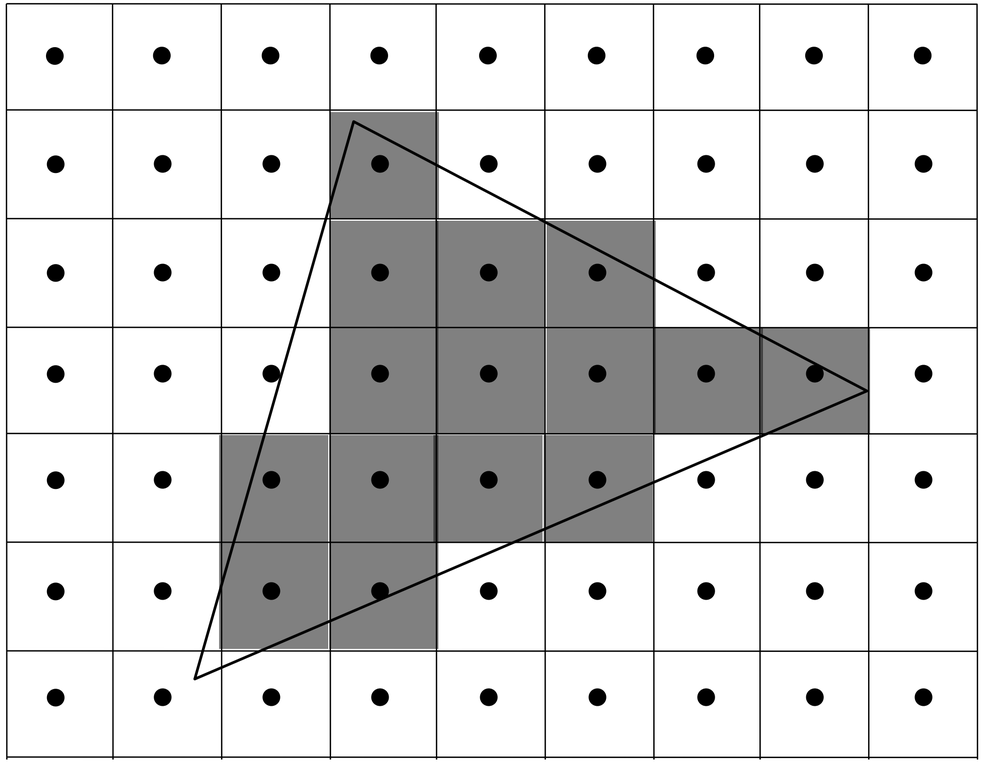


Image Copyright Fritz Kessler

Super Sampling Anti-Aliasing



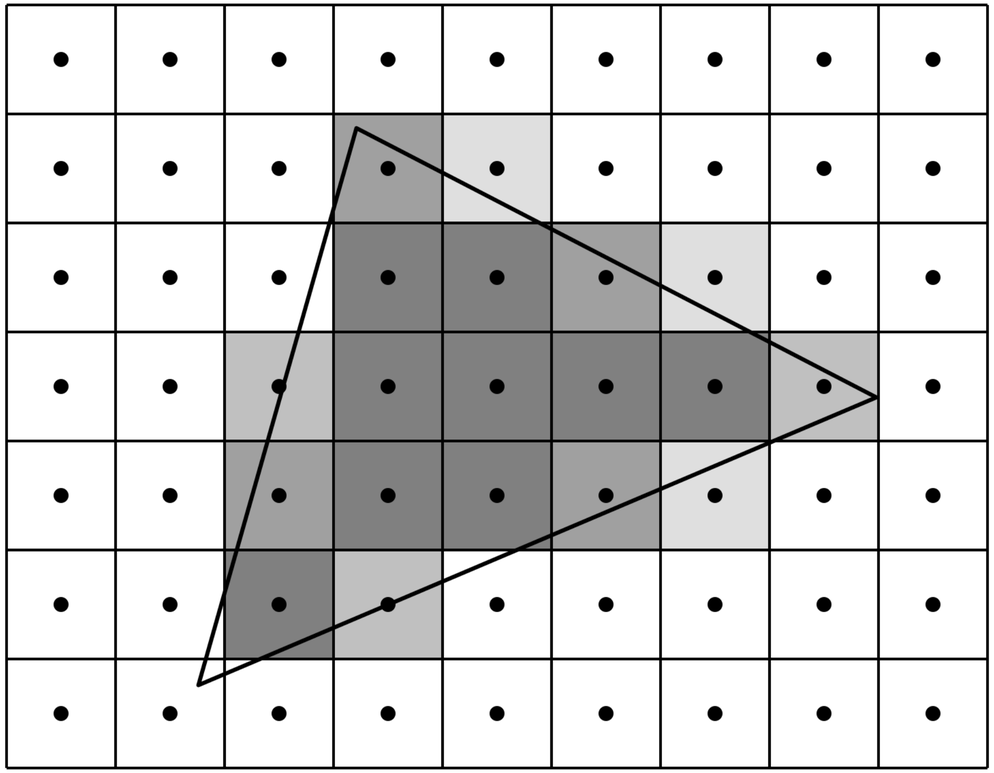
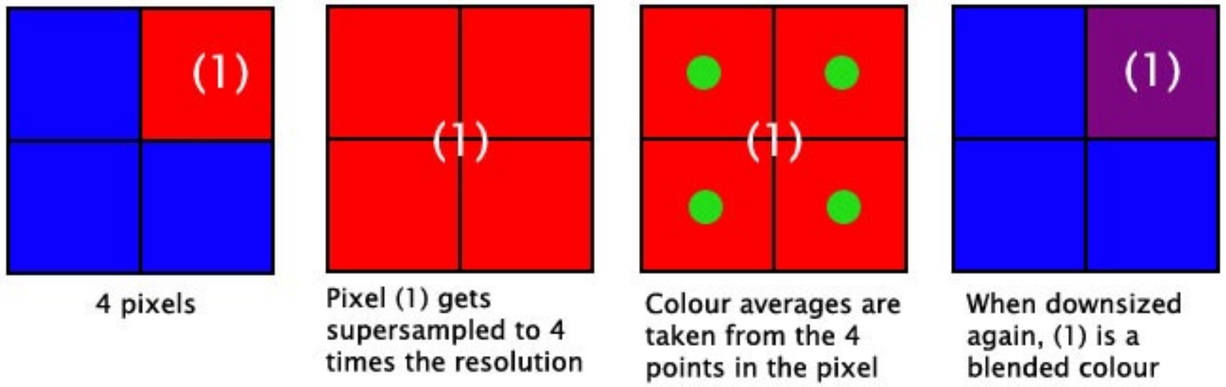


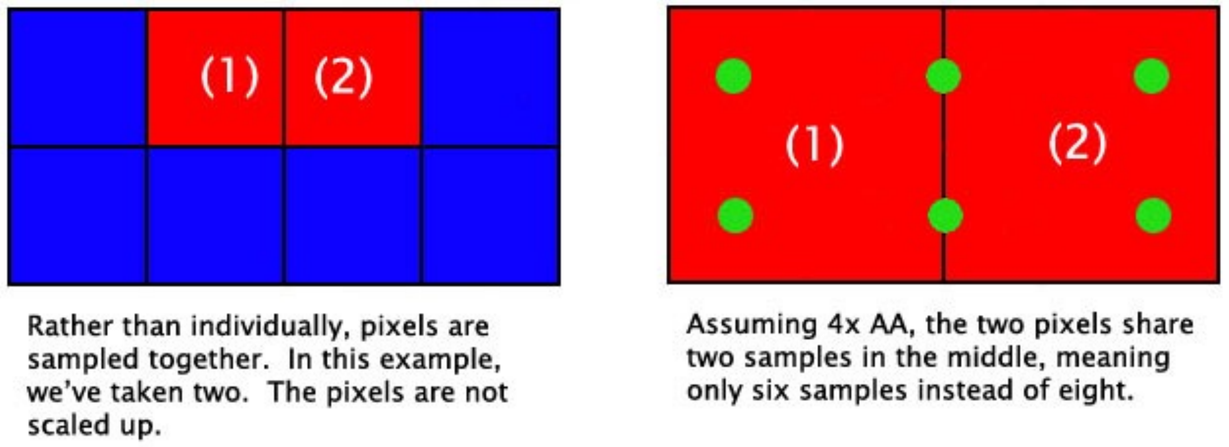
Image Copyright: Fritz Kessler

Many different names and variants

- SSAA (FSAA)
- MSAA
- CSAA
- EQAA
- FXAA
- TXAA



MSAA



Copyright: tested.com (http://www.tested.com/tech/pcs/1194-how-to-choose-the-right-anti-aliasing-mode-for-your-gpu/#)

References

Fundamentals of Computer Graphics, Fourth Edition
4th Edition **by Steve Marschner, Peter Shirley**

Chapter 8